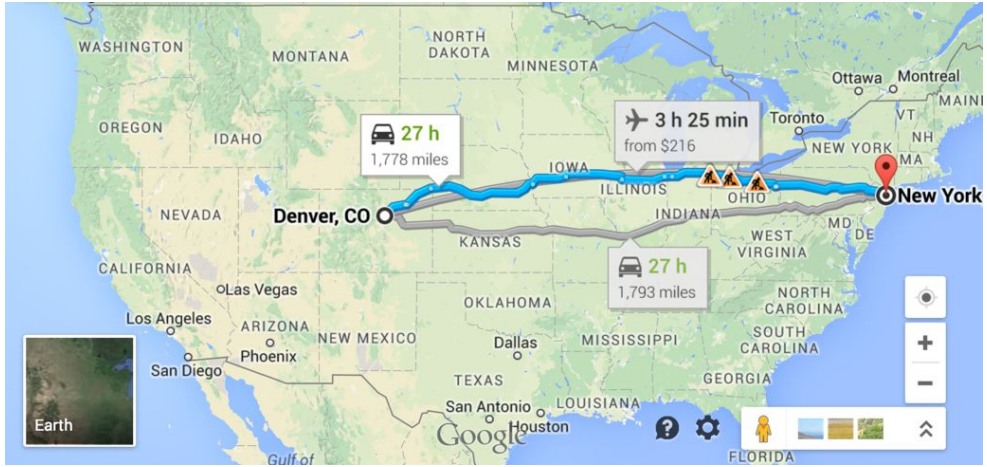


Lecture 23 (Graphs 3)

Shortest Paths

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Shortest Paths: Why BFS Doesn't Work

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- **Why BFS Doesn't Work**
- Goal: The Shortest Paths Tree

Dijkstra's Algorithm

- Dijkstra's Algorithm
- Runtime Analysis

A* (Extra, not in scope)

- A* Idea and Demo
- A* Heuristics (CS188 Preview)

Graph Problems

Problem	Problem Description	Solution	Efficiency (adj. list)
s-t paths	Find a path from s to every reachable vertex.	DepthFirstPaths.java Demo	O(V+E) time Θ(V) space
s-t shortest paths	Find a shortest path from s to every reachable vertex.	BreadthFirstPaths.java Demo	O(V+E) time Θ(V) space

Last time, saw two ways to find paths in a graph.

- DFS and BFS.

Which is better?

BFS vs. DFS for Path Finding

Possible considerations:

- **Correctness.** Do both work for all graphs?
  - Yes!
- **Output Quality.** Does one give better results?
  - BFS is a 2-for-1 deal, not only do you get paths, but your paths are also guaranteed to have the fewest edges.
- **Time Efficiency.** Is one more efficient than the other?
  - Should be very similar. Both consider all edges twice. Experiments or very careful analysis needed.

BFS vs. DFS for Path Finding

Space Efficiency. Is one more efficient than the other?

- DFS is worse for spindly graphs.
  - Call stack gets very deep.
  - Computer needs Θ(V) memory to remember recursive calls (see CS61C).
- BFS is worse for absurdly “bushy” graphs.
  - Queue gets very large. In worst case, queue will require Θ(V) memory.
  - Example: 1,000,000 vertices that are all connected. 999,999 will be enqueued at once.
- Note: In our implementations, we have to spend Θ(V) memory anyway to track distTo and edgeTo arrays.
  - Can optimize by storing distTo and edgeTo in a map instead of an array.

BreadthFirstSearch for Google Maps

Observation: BFS would not be a good choice for a google maps style navigation application.

- The problem: BFS returns path with shortest number of edges, not necessarily the shortest path.

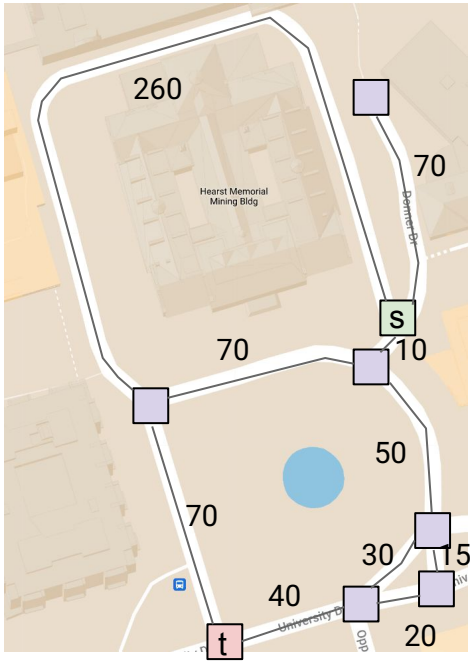
Let’s see a quick example.

Breadth First Search for Mapping Applications

Suppose we're trying to get from s to t.

Breadth First Search for Mapping Applications

Suppose we're trying to get from s to t.

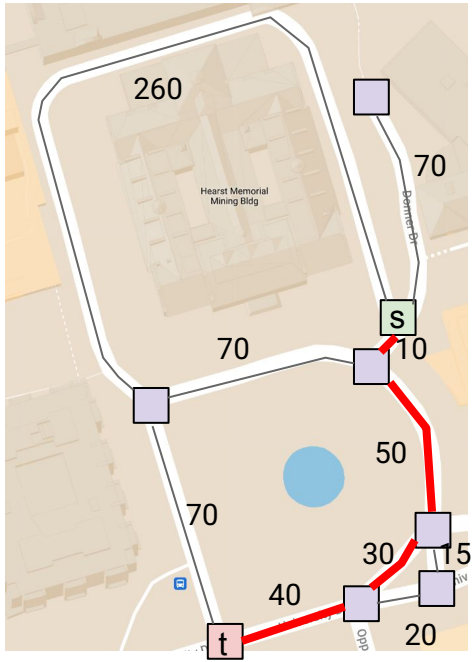


Breadth First Search for Mapping Applications

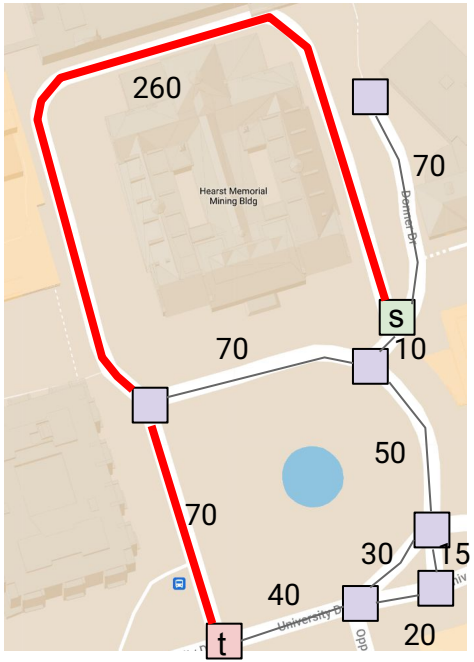
BFS yields the wrong route from s to t.

- No: BFS yields a route of length ~330 m instead of ~130 m.
- We need an algorithm that takes into account edge distances, also known as “edge weights”!

Correct Result



BFS Result



Goal: The Shortest Paths Tree

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- Why BFS Doesn't Work
- **Goal: The Shortest Paths Tree**

Dijkstra's Algorithm

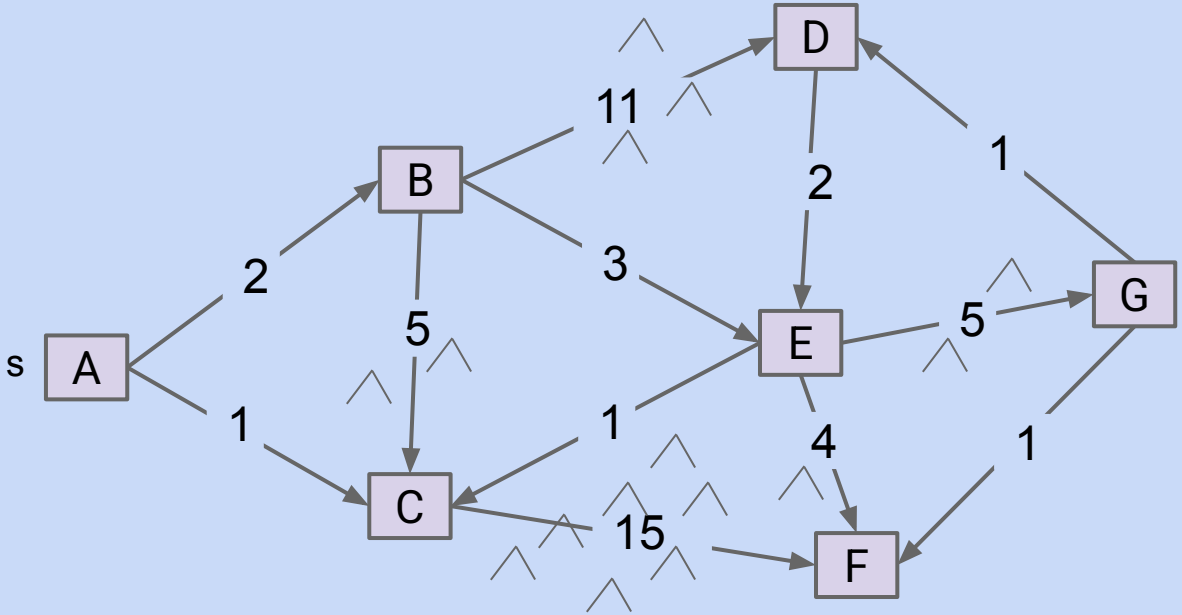
- Dijkstra's Algorithm
- Runtime Analysis

A* (Extra, not in scope)

- A* Idea and Demo
- A* Heuristics (CS188 Preview)

Problem: Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t.

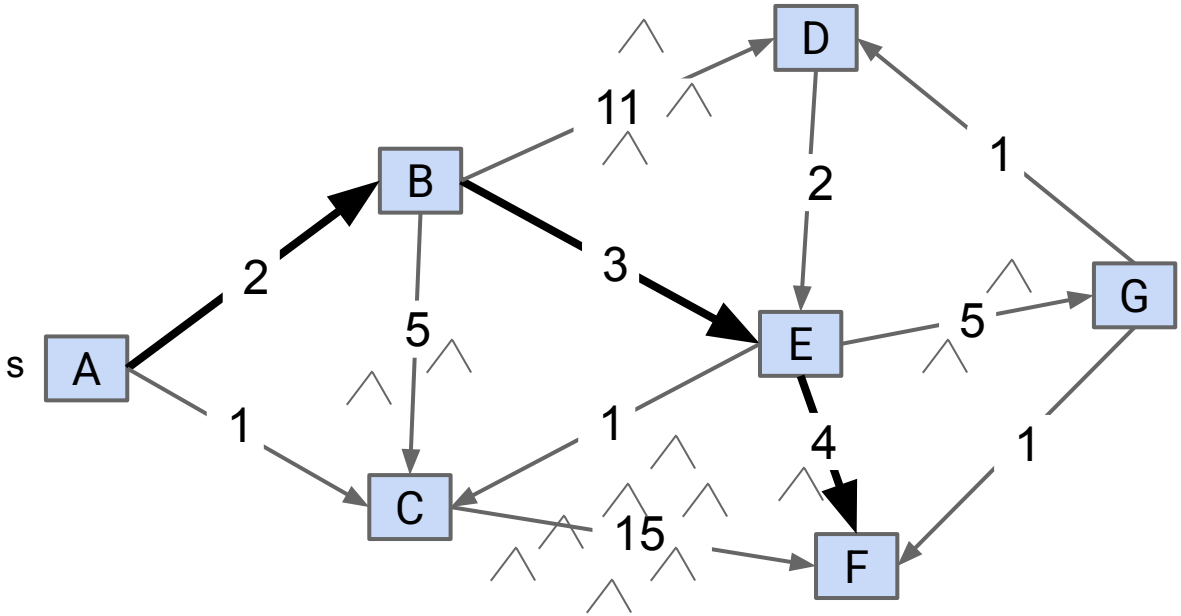


Challenge: Try to find the shortest path from town A to town F.

- Each edge has a number representing the length of that road in miles.

Problem: Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t.



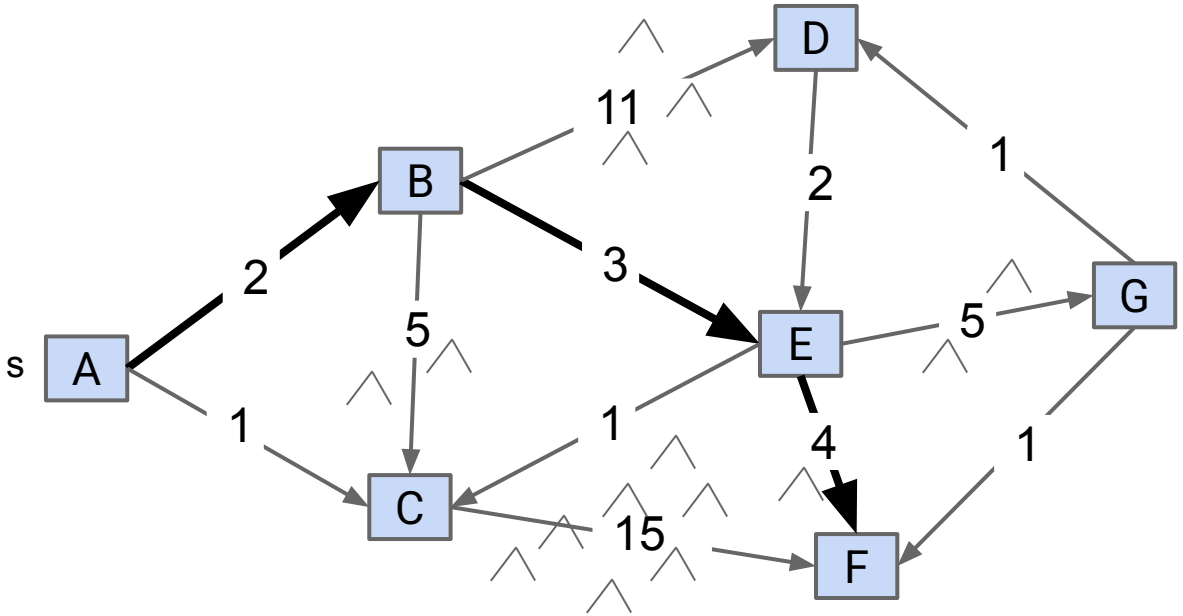
Best path from A to F is

- A -> B -> E -> F.
- Total length is 9 miles.

The path A -> C -> F only involves three towns, but total length is 16 miles.

Problem: Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t.



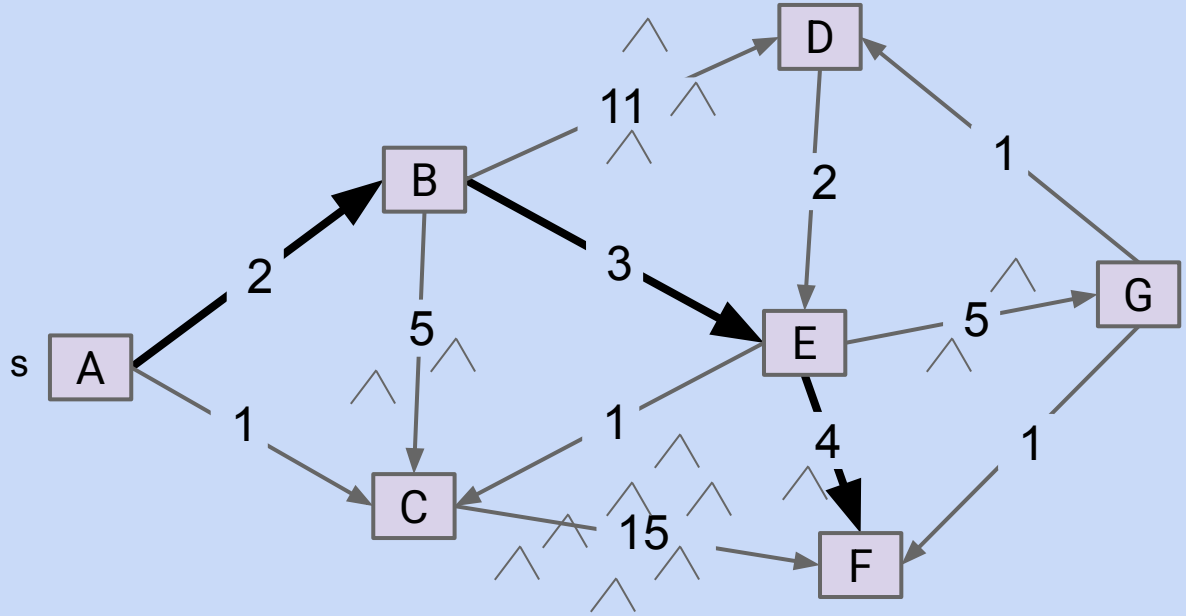
v	distTo[]	edgeTo[]
A	0.0	-
B	2.0	A→B
C	-	-
D	-	-
E	5.0	B→E
F	9.0	E→F
G	-	-

Shortest path from s=A to t=F

Observation: Solution will always be a path with no cycles (assuming non-negative weights).

Problem: Single Source Shortest Paths

Goal: Find the shortest paths from source vertex s to every other vertex.

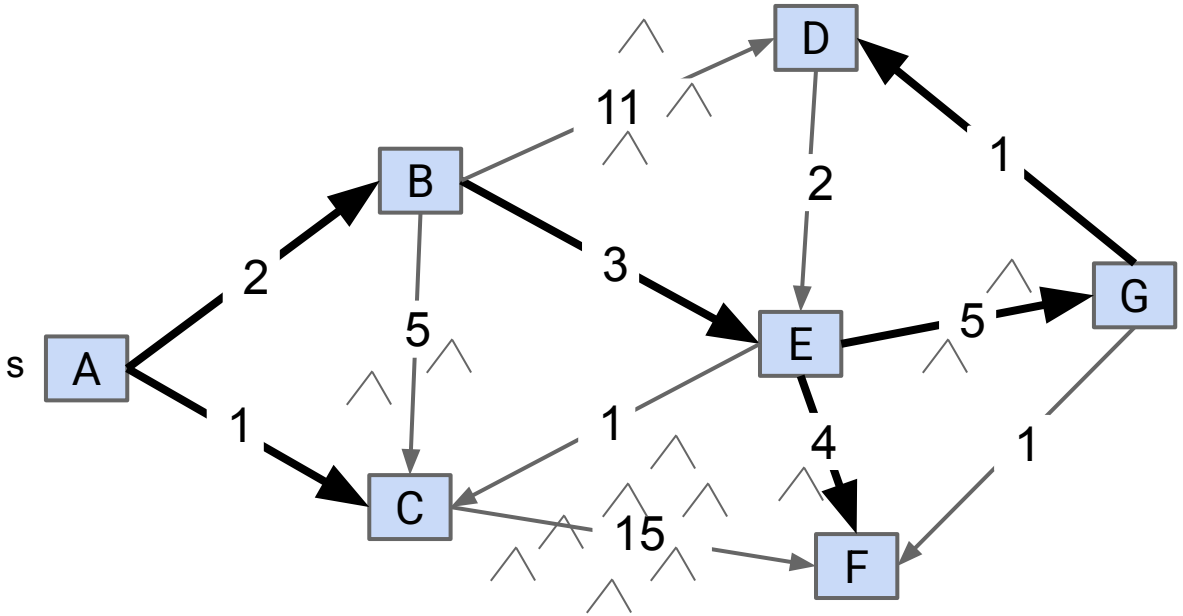


Challenge: Try to write out the solution for this graph.

- You should notice something interesting.

Problem: Single Source Shortest Paths

Goal: Find the shortest paths from source vertex s to every other vertex.



v	distTo[]	edgeTo[]
A	0.0	-
B	2.0	A→B
C	1.0	A→B
D	11.0	G→D
E	5.0	B→E
F	9.0	E→F
G	10.0	E→G

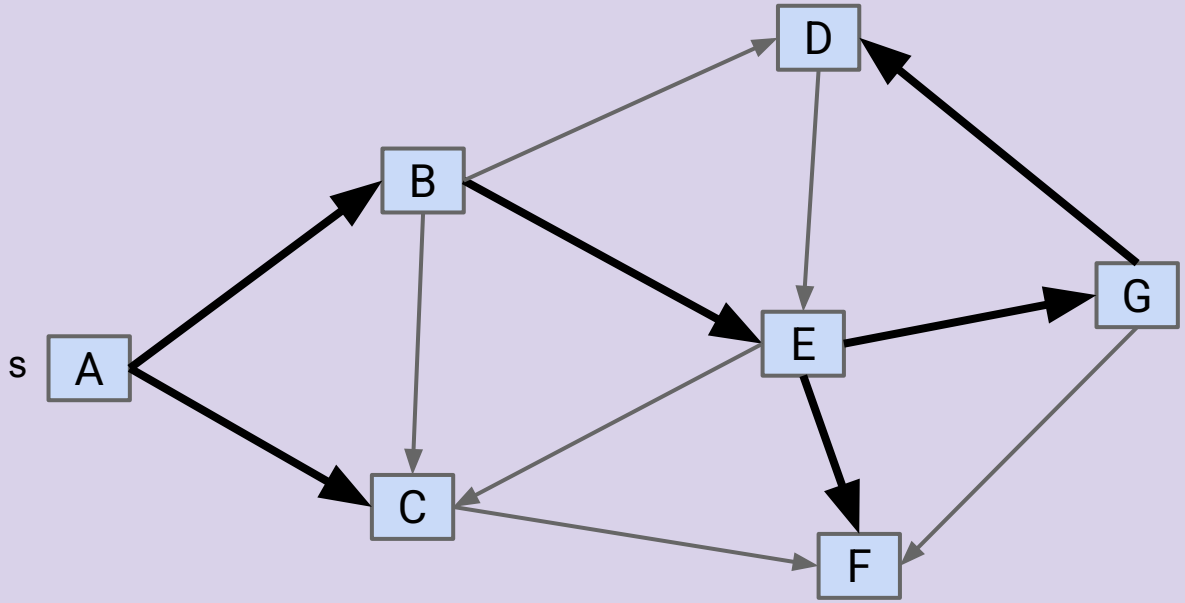
Shortest paths from s=A

Observation: Solution will always be a **tree**.

- Can think of as the union of the shortest paths to all vertices.

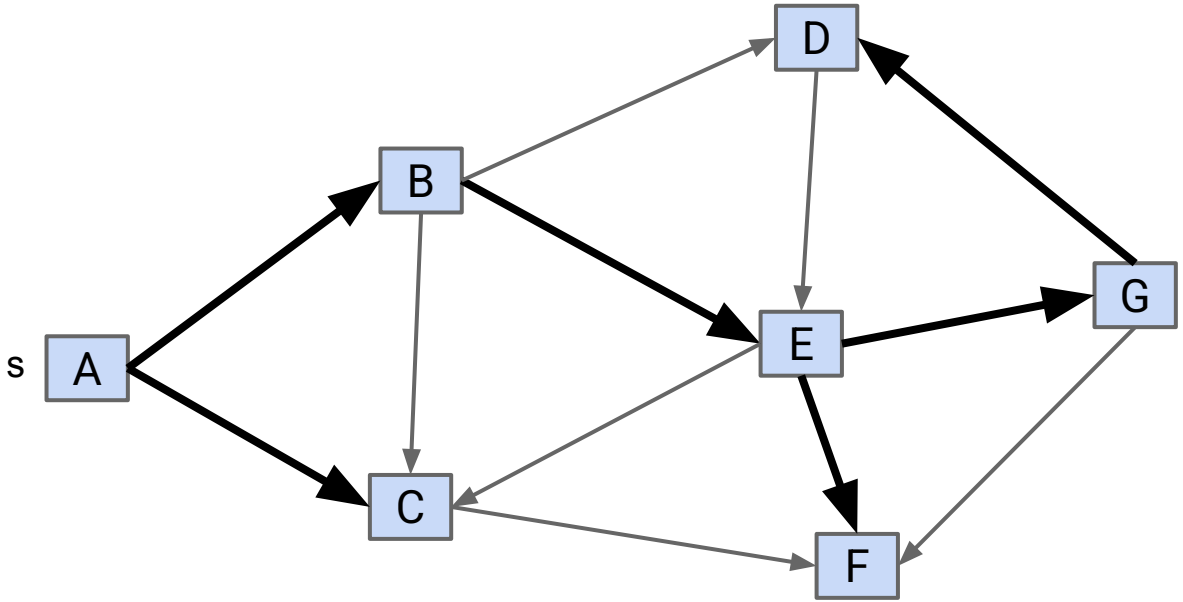
SPT Edge Count: yellkey.com/pull

If G is a connected edge-weighted graph with V vertices and E edges, how many edges are in the **Shortest Paths Tree** (SPT) of G? [assume every vertex is reachable]



SPT Edge Count

If G is a connected edge-weighted graph with V vertices and E edges, how many edges are in the **Shortest Paths Tree** (SPT) of G? [assume every vertex is reachable]



V: 7
Number of edges in SPT is 6

Always V-1:

- For each vertex, there is exactly one input edge (except source).

Edge Relaxation

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- Why BFS Doesn't Work
- Goal: The Shortest Paths Tree

Dijkstra's Algorithm

- Dijkstra's Algorithm
- Runtime Analysis

A* (Extra, not in Scope)

- A* Idea and Demo
- A* Heuristics (CS188 Preview)

Edge Relaxation

There are many algorithms for shortest paths.

One common approach:

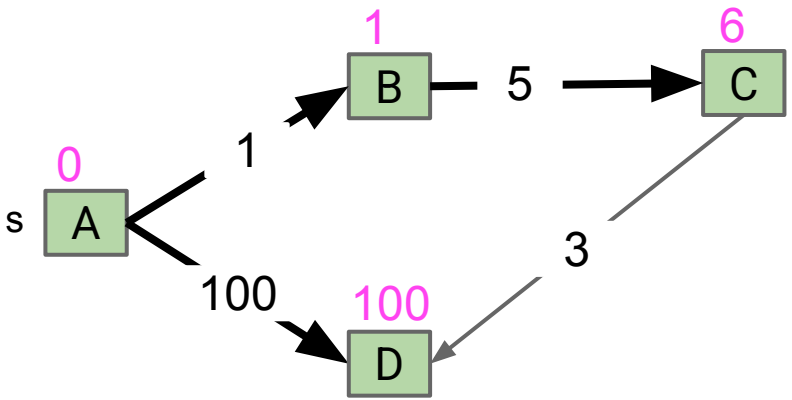
- Start from an incorrect SPT.
- Make improvements until your SPT is correct.

Common improvement operation: **Edge Relaxation**.

- `relax(E)` definition: If edge E is better, use it instead.

Edge Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

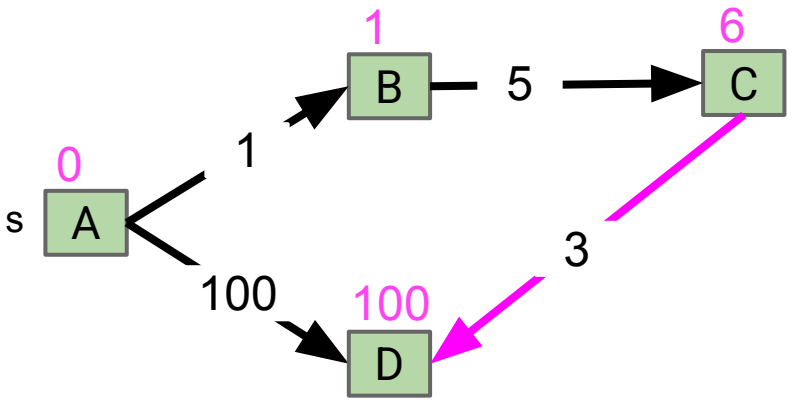


v	distTo[]	edgeTo[]
A	0.0	-
B	1.0	A→B
C	6.0	B→C
D	100.0	A→D

Edge Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax the **magenta** edge, what happens?

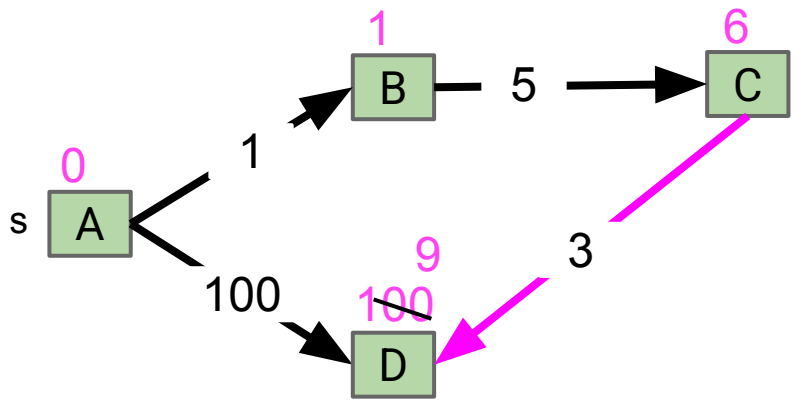


v	distTo[]	edgeTo[]
A	0.0	-
B	1.0	A→B
C	6.0	B→C
D	100.0	A→D

Edge Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax the **magenta** edge, what happens?

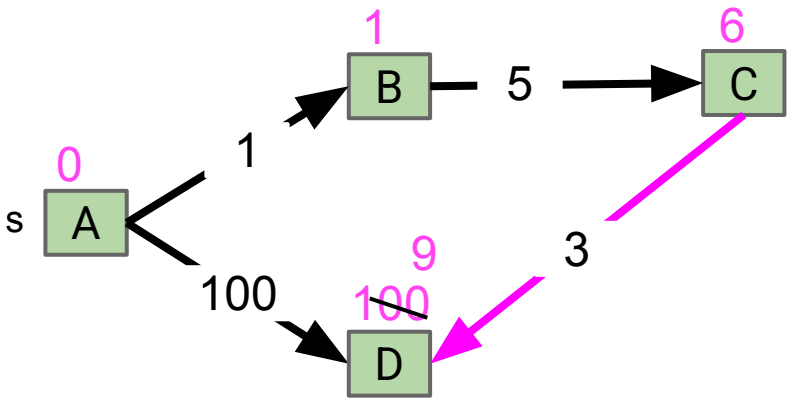


v	distTo[]	edgeTo[]
A	0.0	-
B	1.0	A→B
C	6.0	B→C
D	100.0	A→D

Edge Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

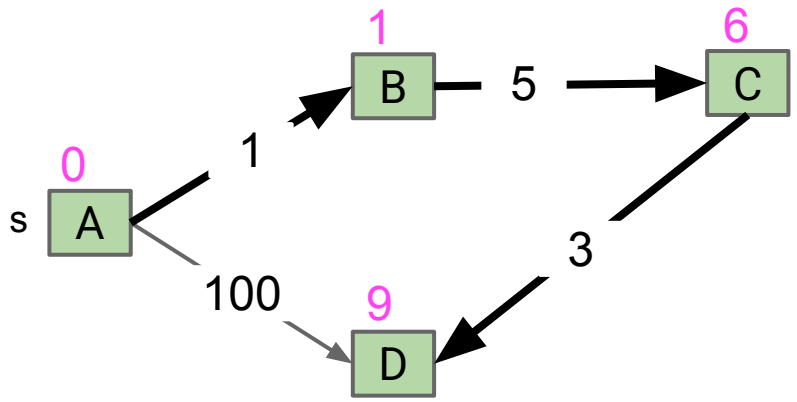
- If we relax the **magenta** edge, what happens?



v	distTo[]	edgeTo[]
A	0.0	-
B	1.0	A→B
C	6.0	B→C
D	100.0	A→D

Edge Relaxation Example

This shortest path tree is now correct.



v	distTo[]	edgeTo[]
A	0.0	-
B	1.0	A→B
C	6.0	B→C
D	9.0	C→D

Node Relaxation

There are many algorithms for shortest paths.

One common approach:

- Start from an incorrect SPT.
- Make improvements until your SPT is correct.

Common improvement operation: **Edge Relaxation**.

- `relax(E)` definition: If edge E is better, use it instead.

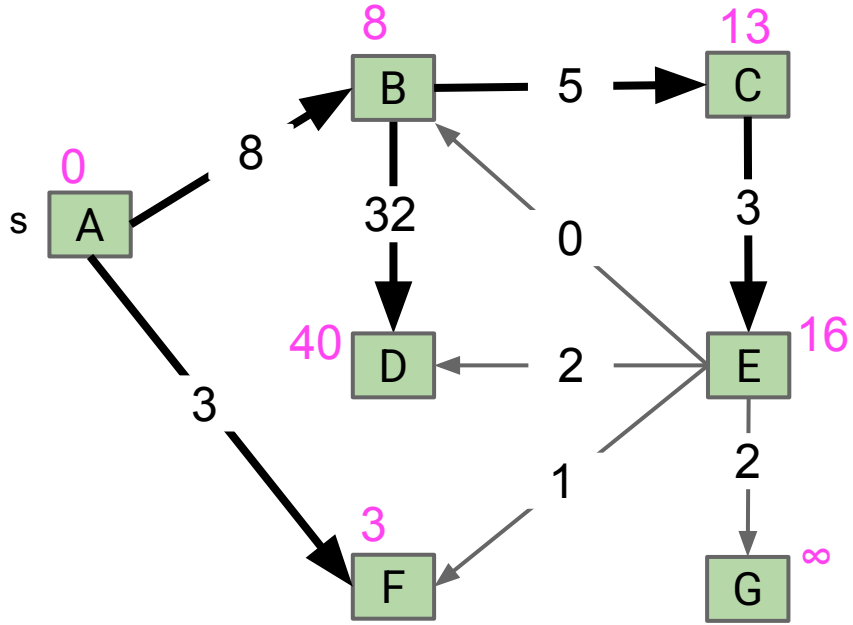
Common improvement operation: **Node Relaxation**

- `relax(v)` definition: Relax all of v's outgoing edges.

Node Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax node E what happens?
- Write the `distTo` and `edgeTo` after relaxation.

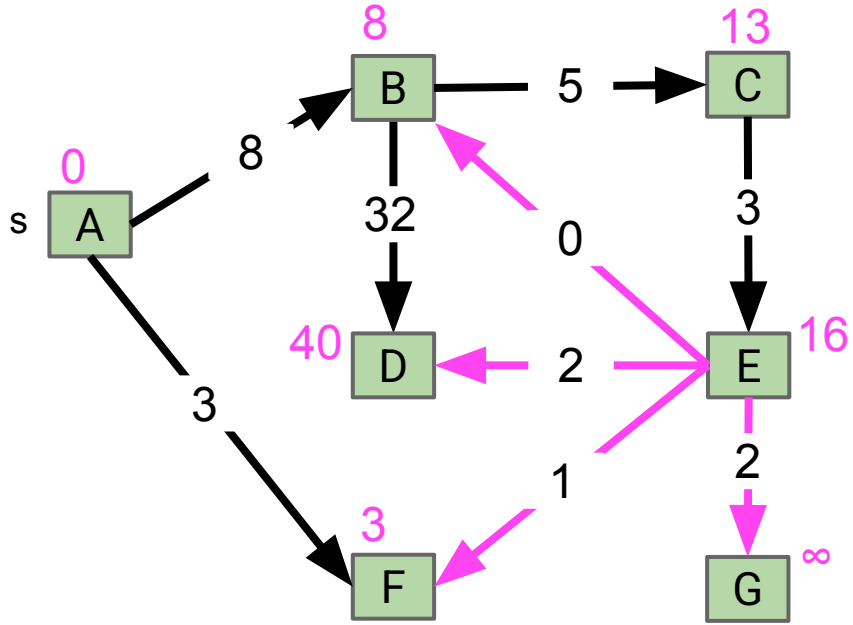


v	distTo[]	edgeTo[]
A	0.0	-
B	8.0	A→B
C	13.0	B→C
D	40.0	B→D
E	16.0	C→E
F	3.0	A→F
G	∞	-

Node Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax node E what happens?
- Write the `distTo` and `edgeTo` after relaxation.

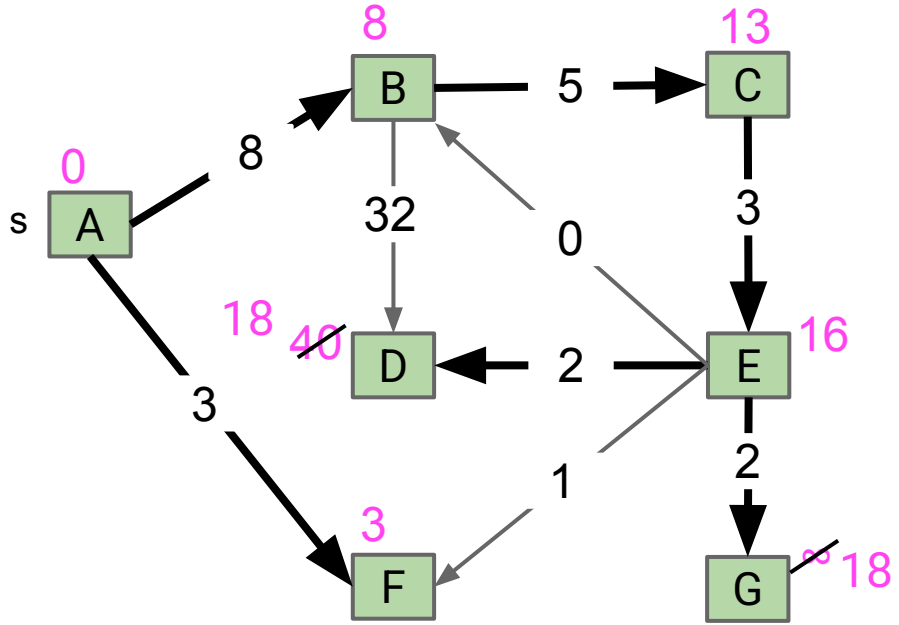


v	distTo[]	edgeTo[]
A	0.0	-
B	8.0	A→B
C	13.0	B→C
D	40.0	B→D
E	16.0	C→E
F	3.0	A→F
G	∞	-

Node Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax node E what happens?
- Write the `distTo` and `edgeTo` after relaxation.

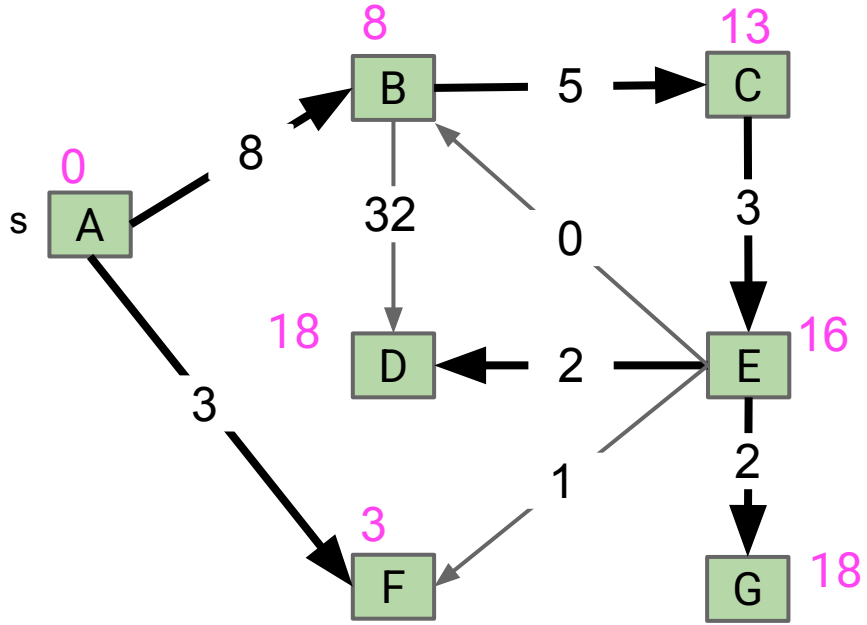


v	distTo[]	edgeTo[]
A	0.0	-
B	8.0	A→B
C	13.0	B→C
D	18.0	E→D
E	16.0	C→E
F	3.0	A→F
G	18.0	E→G

Node Relaxation Example

Suppose we have the incorrect Shortest Paths Tree from A given below.

- If we relax node E what happens?
- Write the `distTo` and `edgeTo` after relaxation.



v	distTo[]	edgeTo[]
A	0.0	-
B	8.0	A→B
C	13.0	B→C
D	18.0	E→D
E	16.0	C→E
F	3.0	A→F
G	18.0	E→G

Dijkstra's Algorithm

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- Why BFS Doesn't Work
- Goal: The Shortest Paths Tree

Dijkstra's Algorithm

- Dijkstra's Algorithm
- Runtime Analysis

A* (Extra, not in Scope)

- A* Idea and Demo
- A* Heuristics (CS188 Preview)

SPT Algorithms

There are many valid SPT algorithms that use relaxation.

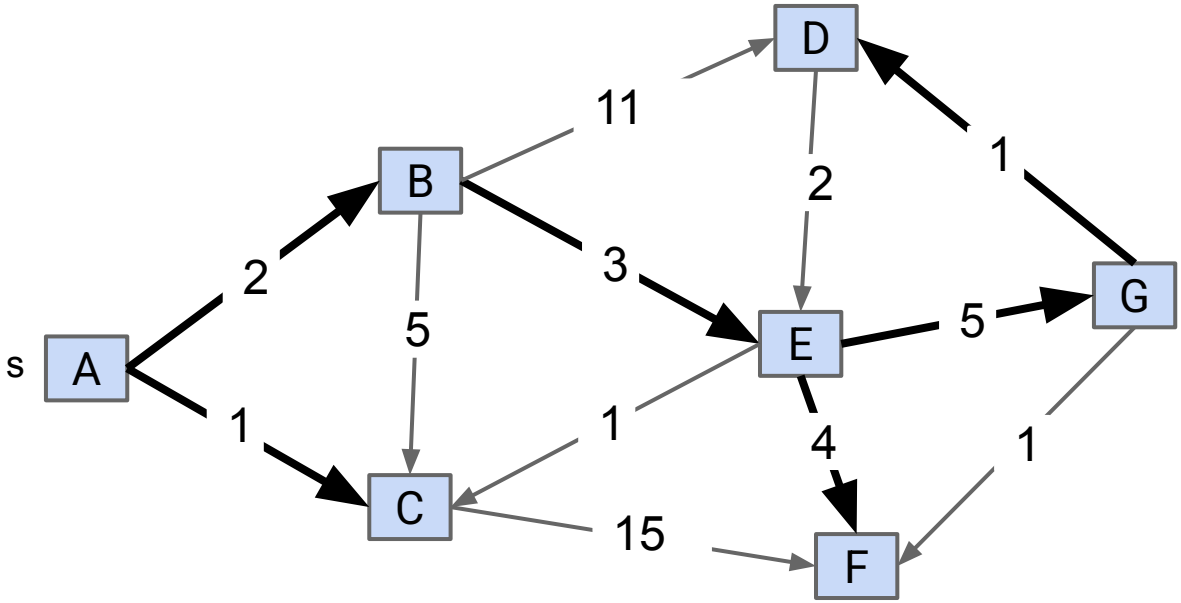
Examples:

- Bellman-Ford (1955): Relax all edges (in any order), repeat V times.
- Dijkstra's Algorithm (1956): Relax all nodes exactly once in "closest first" order.
- The Duan-Mao et. al Algorithm (2025): Relax edges in batches.

We'll only discuss Dijkstra's Algorithm.

Weird Idea: Closest First

We've seen two graph orderings so far: **Depth First** and **Breadth First**.



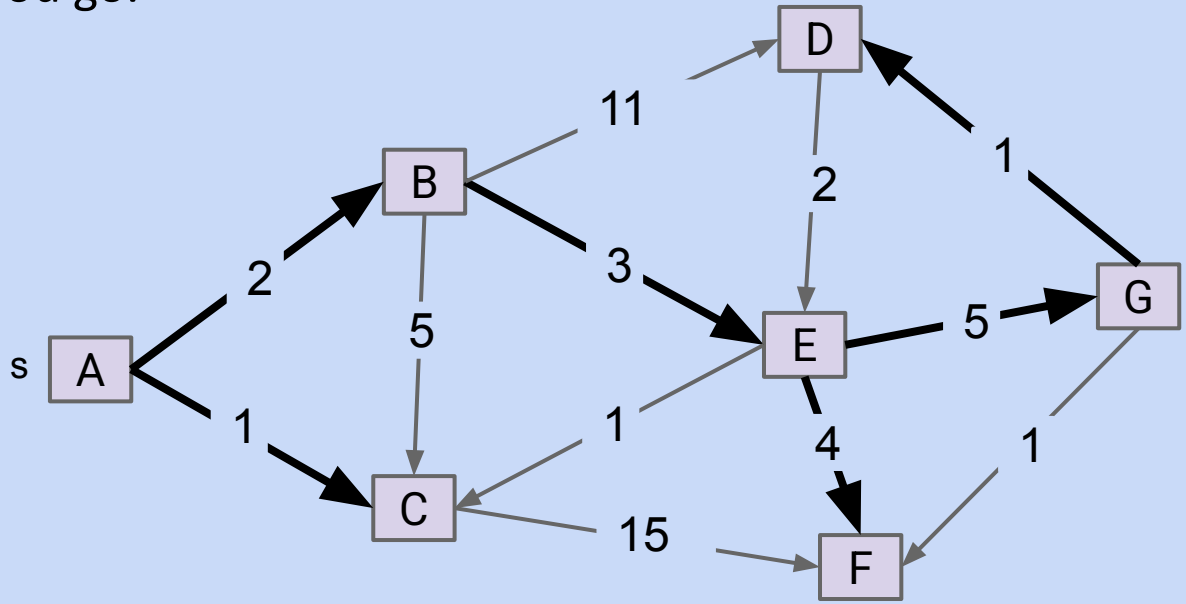
v	distTo[]	edgeTo[]
A	0.0	-
B	2.0	A→B
C	1.0	A→B
D	11.0	G→D
E	5.0	B→E
F	9.0	E→F
G	10.0	E→G

Shortest paths from s=A

Dijkstra's Algorithm: Visit the vertices in **Closest First** order, building the SPT as you go.

Achieving Closest First

Dijkstra's Algorithm: Visit the vertices in **Closest First** order, recording the SPT as you go.



v	distTo[]	edgeTo[]
A	0.0	-
B	2.0	A→B
C	1.0	A→B
D	11.0	G→D
E	5.0	B→E
F	9.0	E→F
G	10.0	E→G

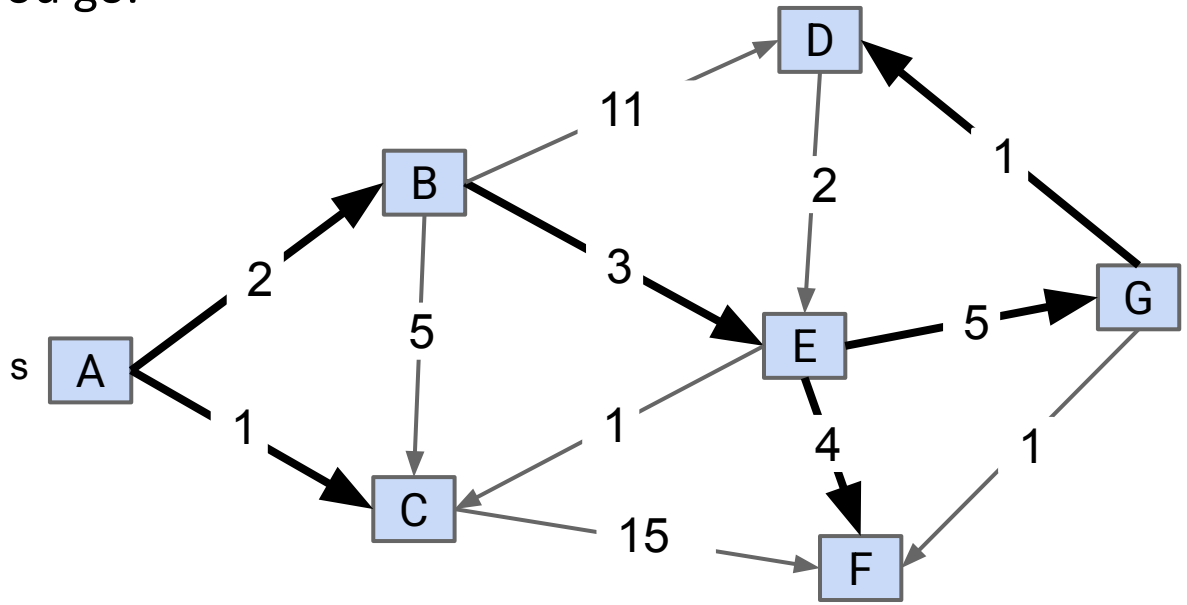
Shortest paths from s=A

Vague question: To achieve Breadth First order, we used a Queue as our fringe.

- If we want **Closest First** order, what data structure should we use as our fringe?

Achieving Closest First

Dijkstra's Algorithm: Visit the vertices in **Closest First** order, recording the SPT as you go.



v	distTo[]	edgeTo[]
A	0.0	-
B	2.0	A→B
C	1.0	A→B
D	11.0	G→D
E	5.0	B→E
F	9.0	E→F
G	10.0	E→G

Shortest paths from s=A

Vague question: To achieve Breadth First order, we used a Queue as our fringe.

- If we want **Closest First** order, what data structure should we use as our fringe? A priority queue.

Achieving Closest First

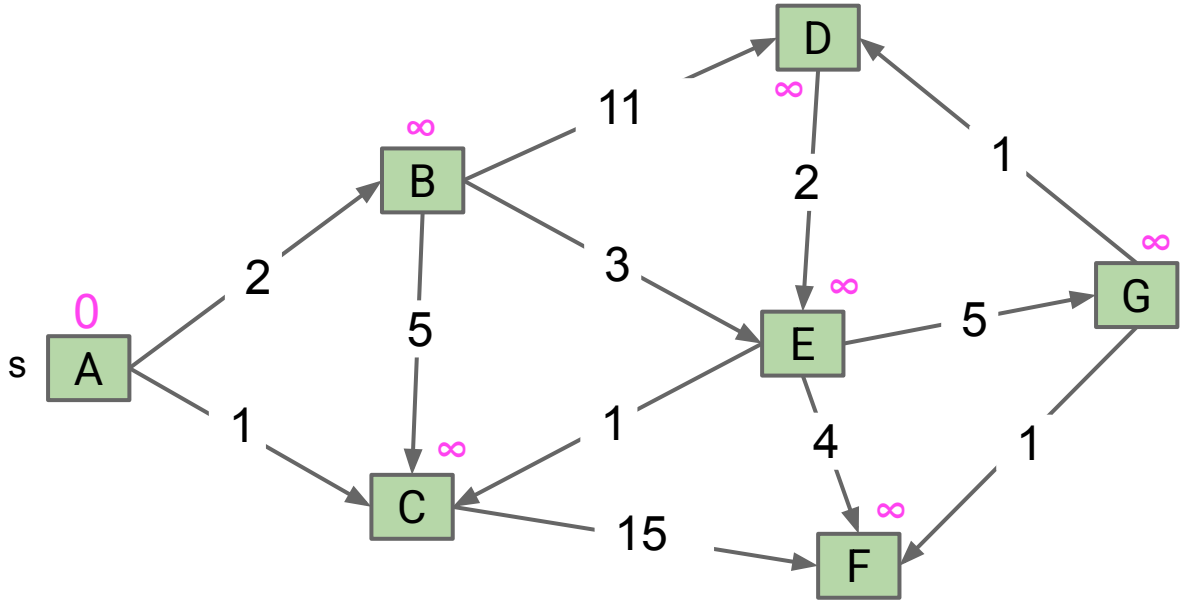
Dijkstra's Algorithm: Visit the vertices in **Closest First** order, recording the SPT as you go. Pseudocode below:

```
add all vertices to PQ with infinite priority
while (!pq.isEmpty()) {
    v = pq.removeMin();
    relax(v); // i.e. relax all edges from v
}
```

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

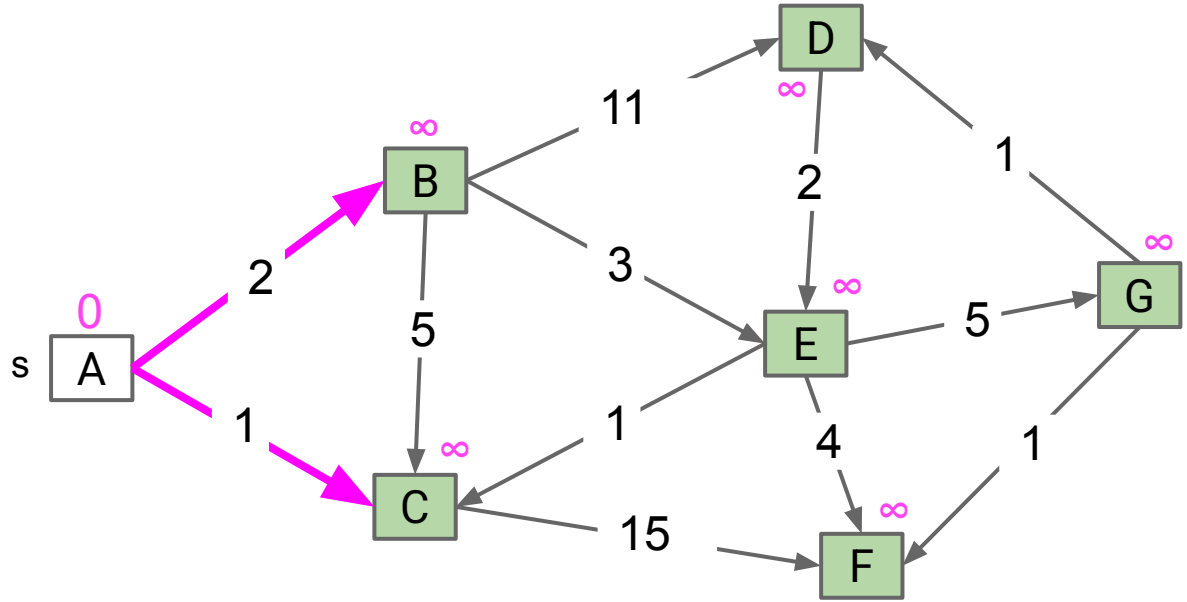


Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-



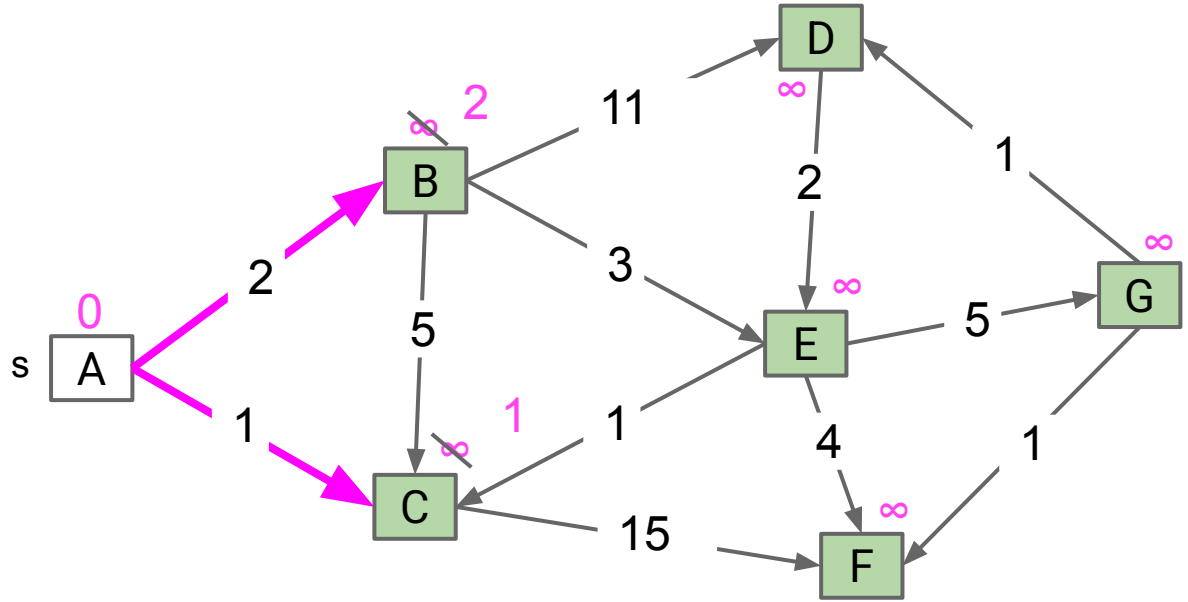
Fringe: [(B: ∞), (C: ∞), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



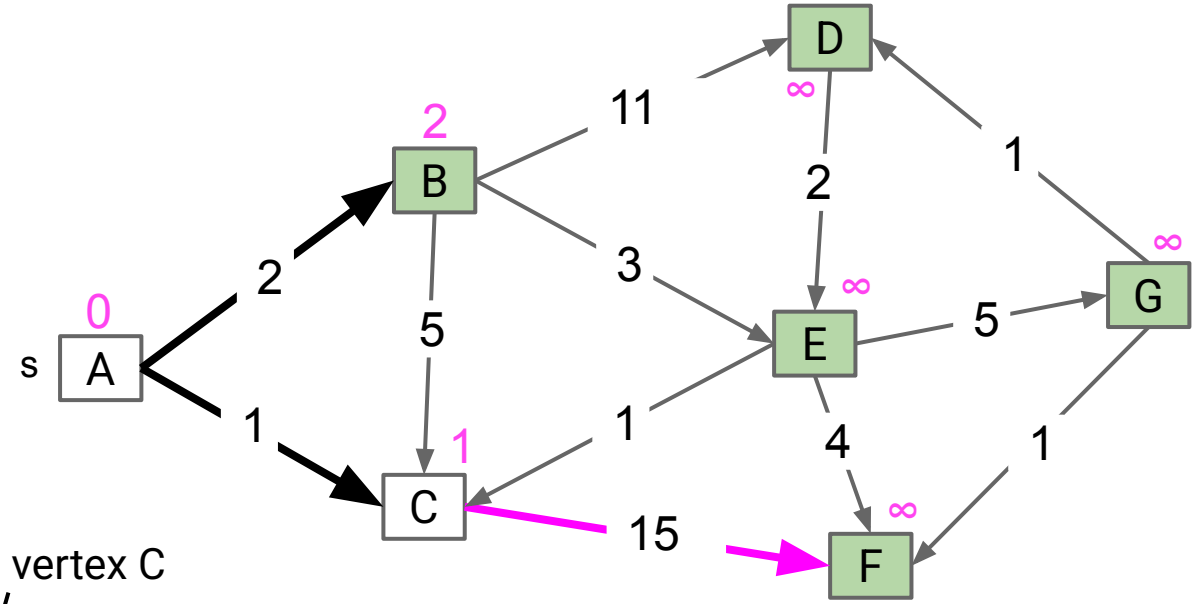
Fringe: [(C: 1), (B: 2), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



Remove vertex C

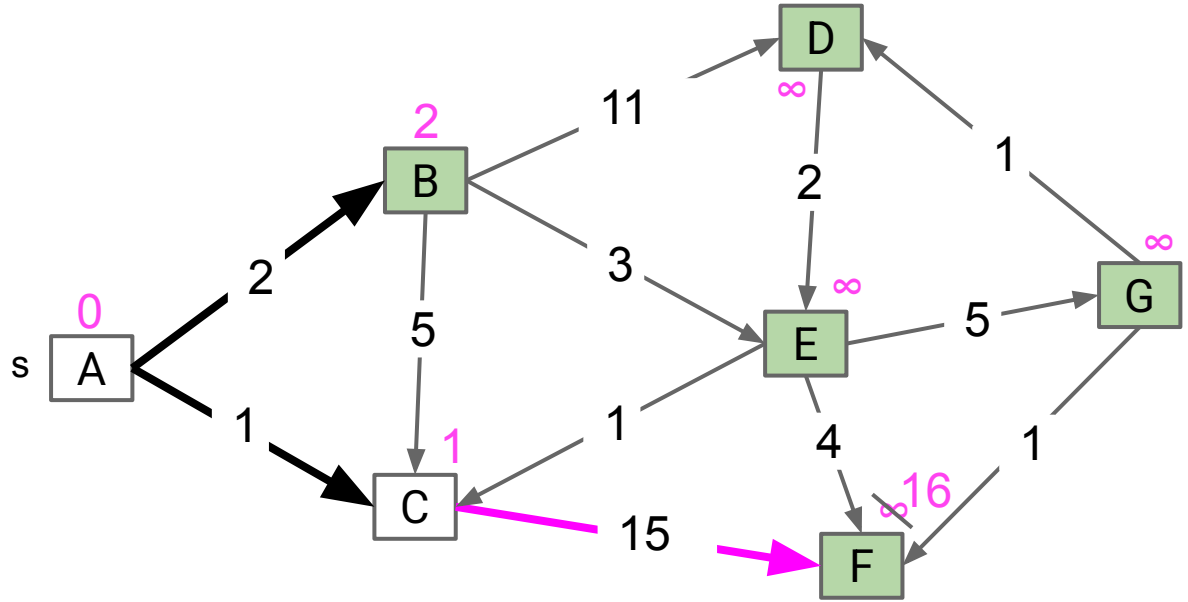
Fringe: [(C: 1), (B: 2), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-



Fringe: [(B: 2), (F: 16), (D: ∞), (E: ∞), (G: ∞)]

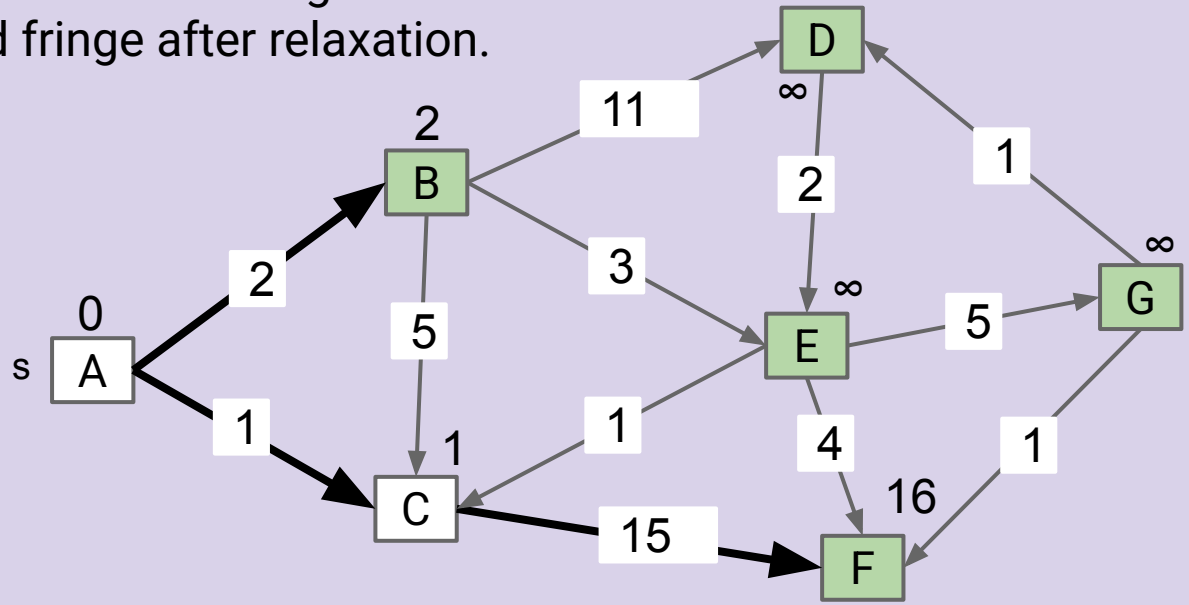
Dijkstra's Demo: yellkey.com/life

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

- What vertex do we remove from the fringe?
- Show distTo, edgeTo, and fringe after relaxation.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-



Fringe: [(B: 2), (F: 16), (D: ∞), (E: ∞), (G: ∞)]

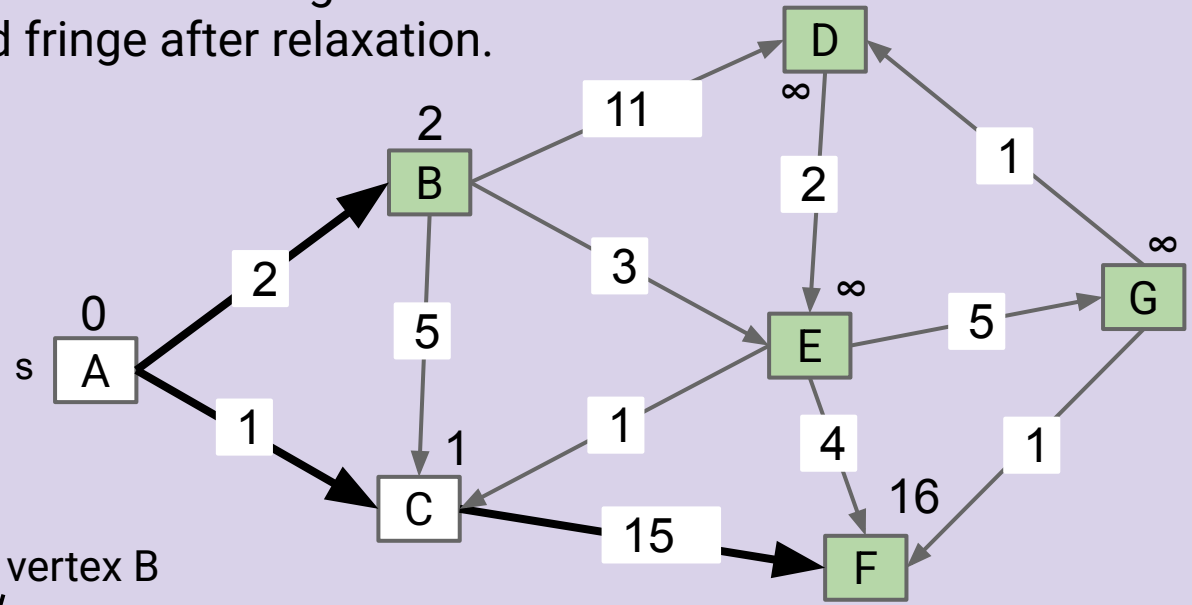
Dijkstra's Demo: yellkey.com/life

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

- What vertex do we remove from the fringe?
- Show distTo, edgeTo, and fringe after relaxation.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-



Remove vertex B

Fringe: [(B: 2), (F: 16), (D: ∞), (E: ∞), (G: ∞)]

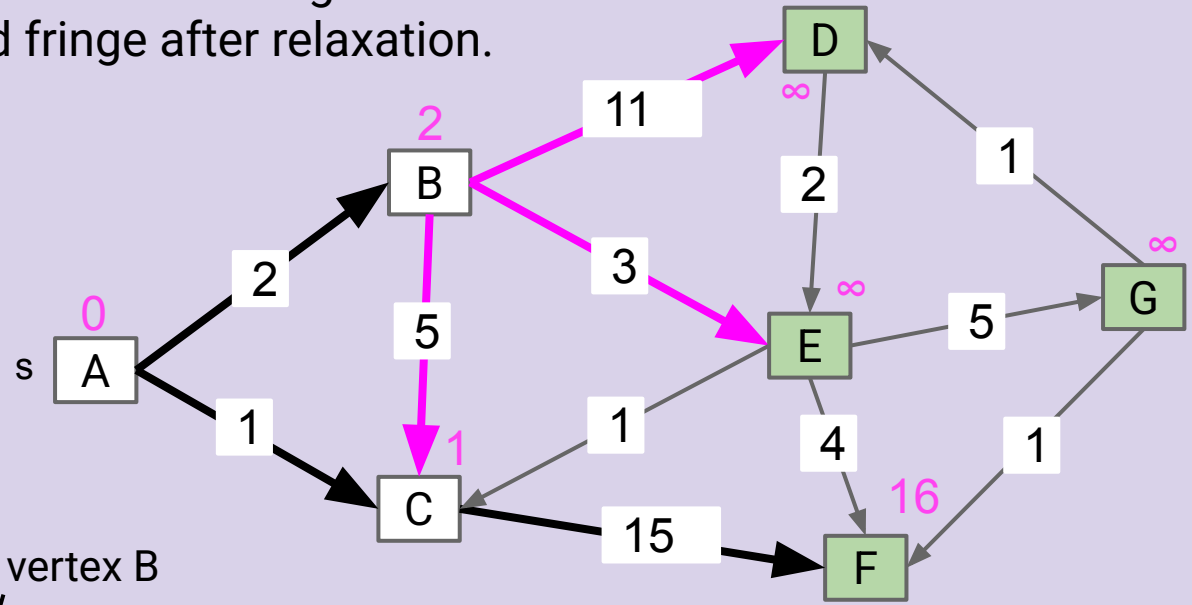
Dijkstra's Demo: yellkey.com/life

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

- What vertex do we remove from the fringe?
- Show distTo, edgeTo, and fringe after relaxation.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-



Remove vertex B

Fringe: [(B: 2), (F: 16), (D: ∞), (E: ∞), (G: ∞)]

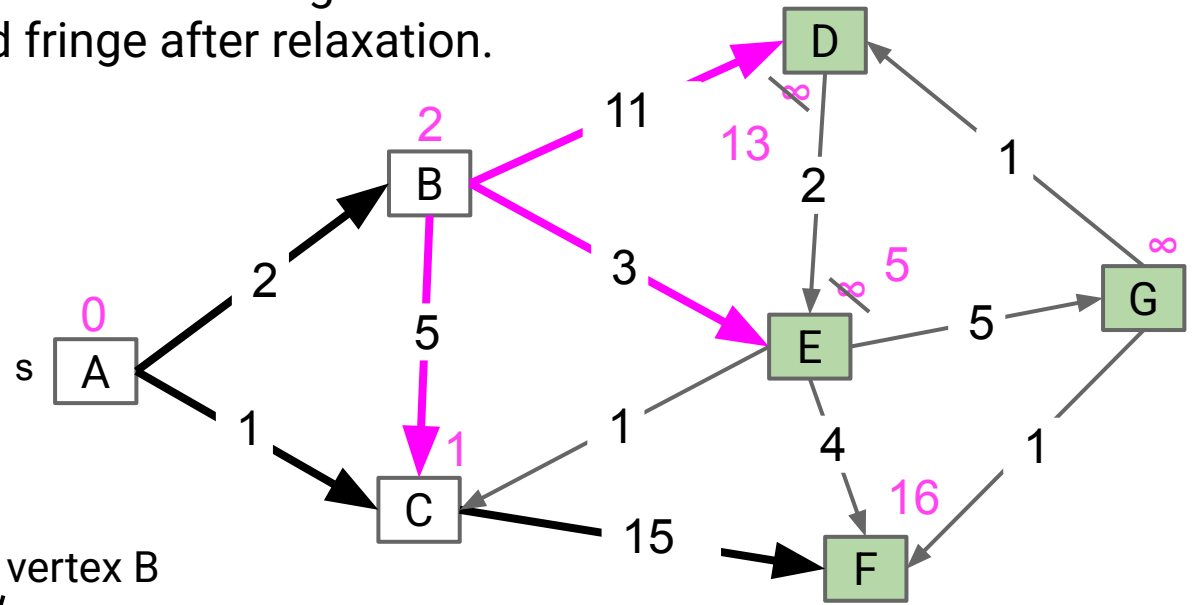
Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

- What vertex do we remove from the fringe?
- Show distTo, edgeTo, and fringe after relaxation.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	~~∞~~ 13	~~-~~ B
E	~~∞~~ 5	~~-~~ B
F	16	C
G	∞	-



Remove vertex B

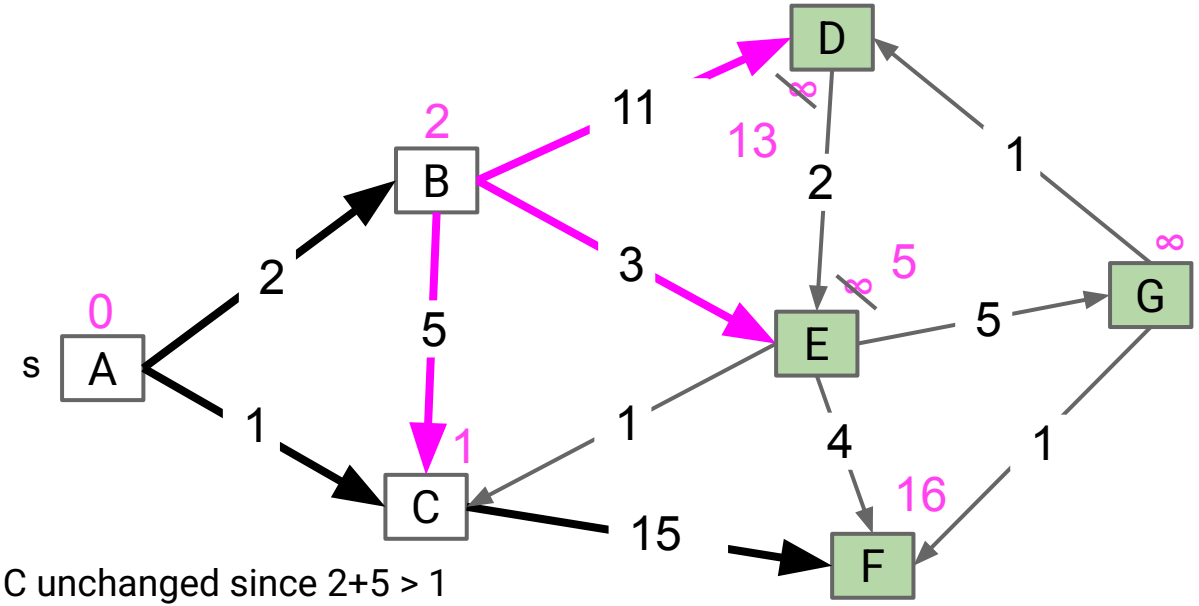
Fringe: [(B: ~~2~~), (F: 16), (D: ~~∞~~ 13), (E: ~~∞~~ 5), (G: ∞)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	16	C
G	∞	-



Vertex C unchanged since 2+5 > 1

Fringe: [(E: 5), (D: 13), (F: 16), (G: ∞)]

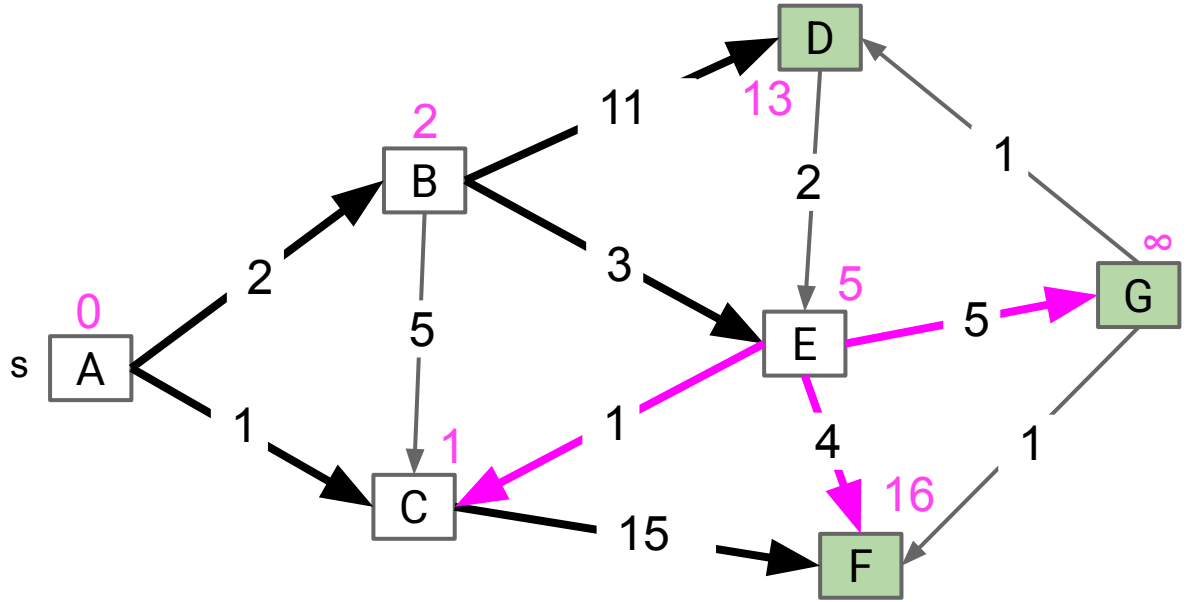
Which vertex is removed next?

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	16	C
G	∞	-

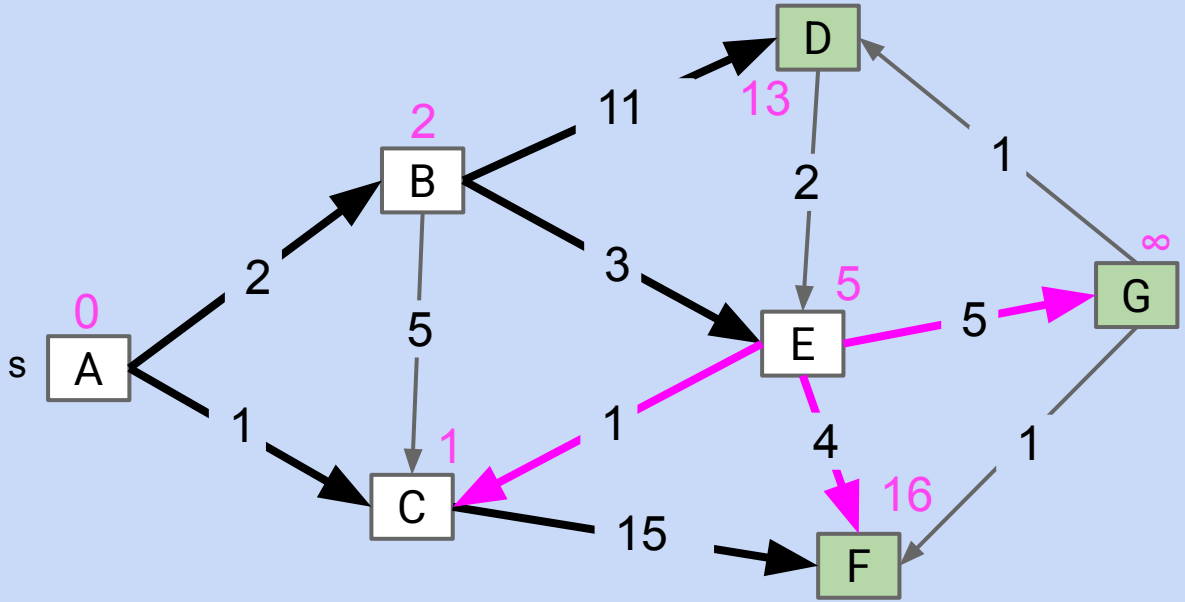


Fringe: [(D: 13), (F: 16), (G: ∞)]

An Interesting Invariant

In Dijkstra's algorithm, relaxing a magenta arrow pointing to a white (already visited) node NEVER works.

Why?

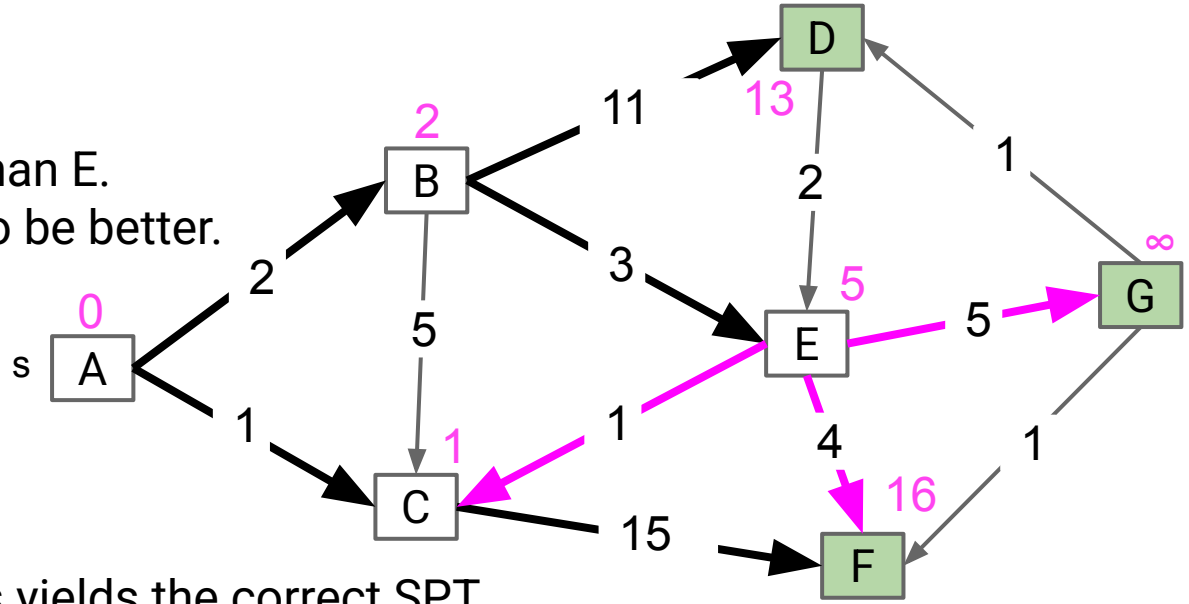


An Interesting Invariant

In Dijkstra's algorithm, relaxing a magenta arrow pointing to a white (already visited) node NEVER works.

Why? By example:

- We already visited C!
- That means C is closer than E.
- So impossible for E→C to be better.



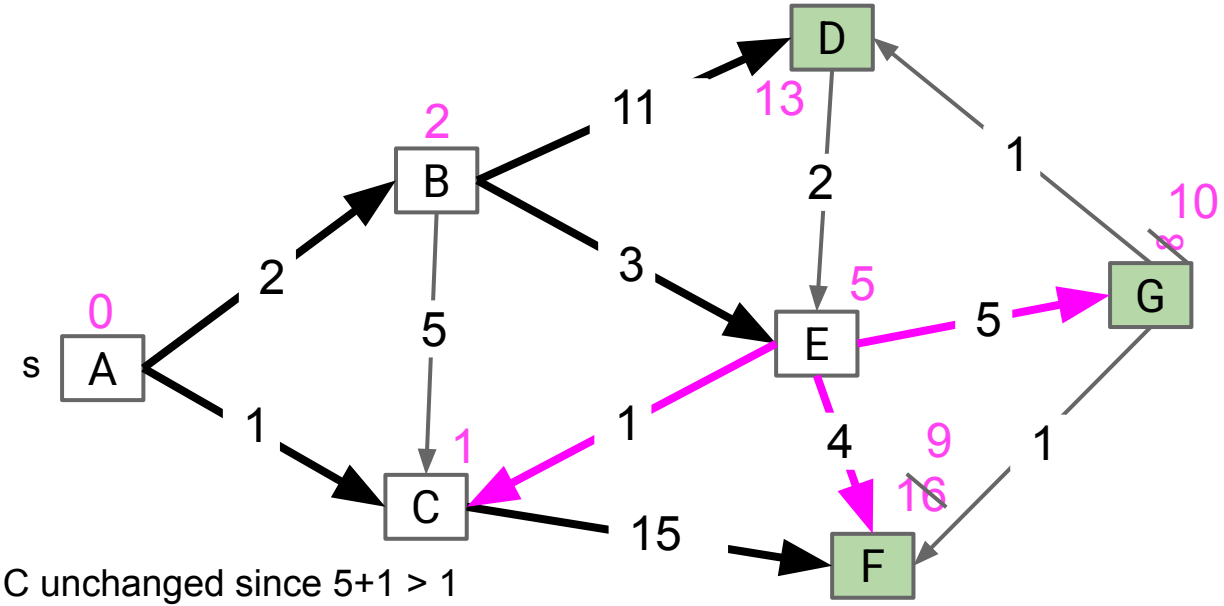
This invariant is why Dijkstra's yields the correct SPT.

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	10	E



Vertex C unchanged since 5+1 > 1

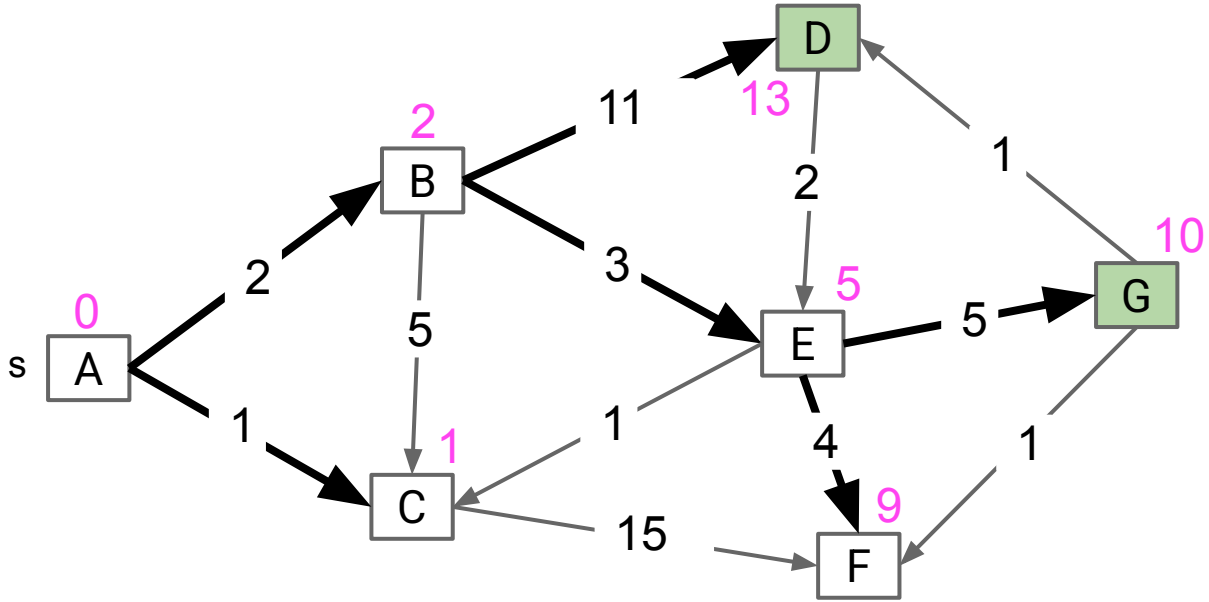
Fringe: [(F: 9), (G: 10), (D: 13)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	10	E



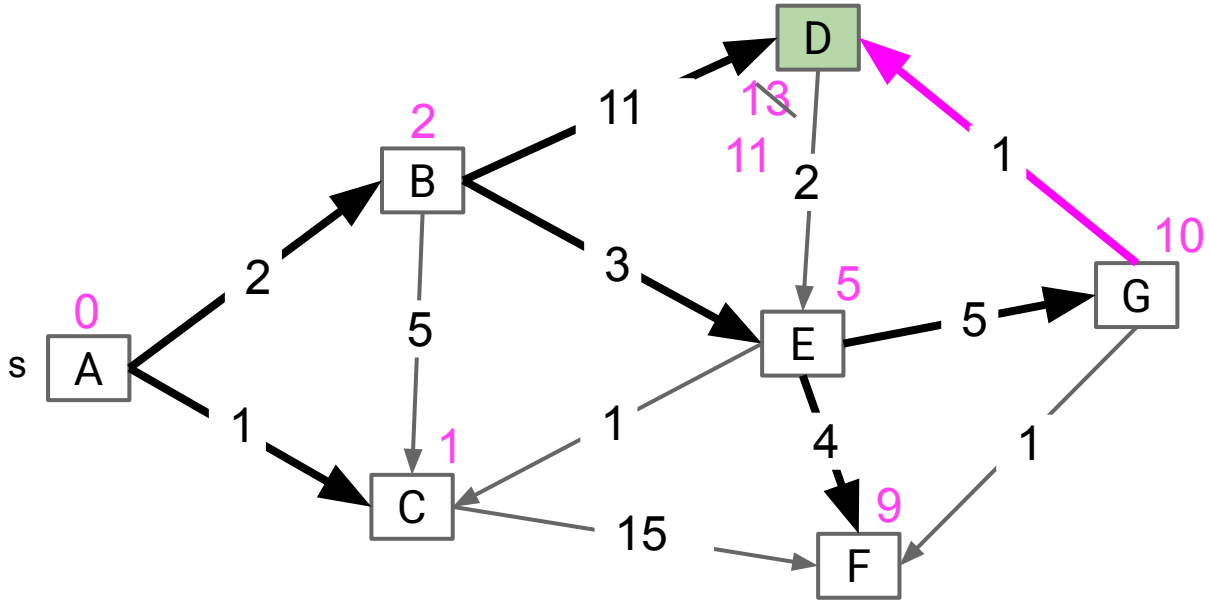
Fringe: [(G: 10), (D: 13)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E



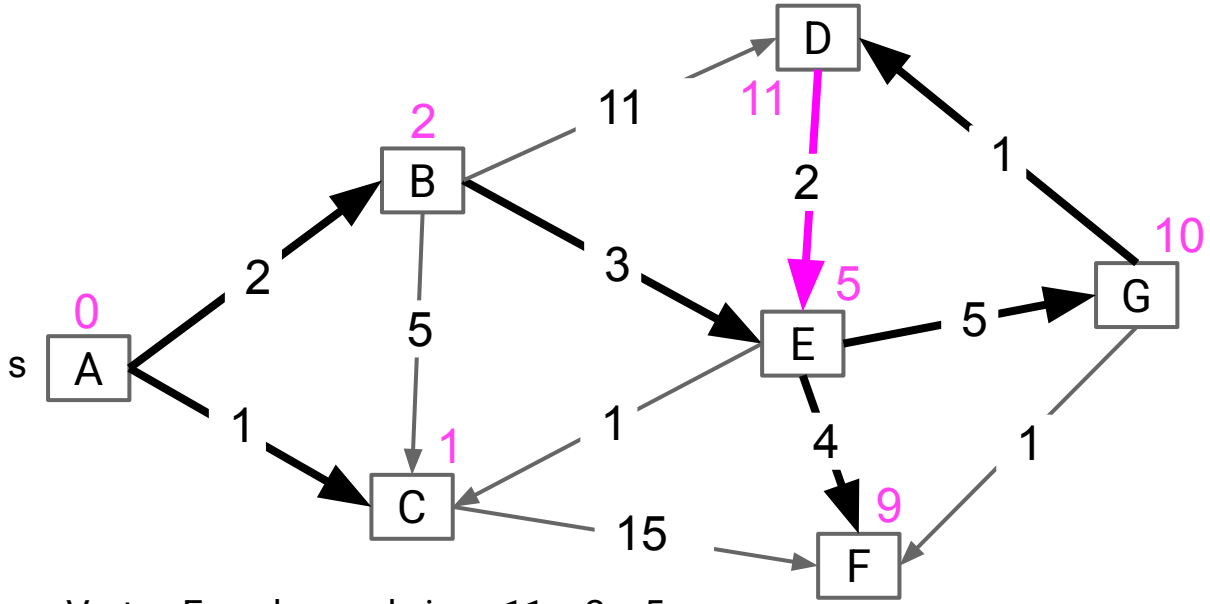
Fringe: [(D: 11)]

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E



Fringe: []

Vertex E unchanged since 11 + 2 > 5

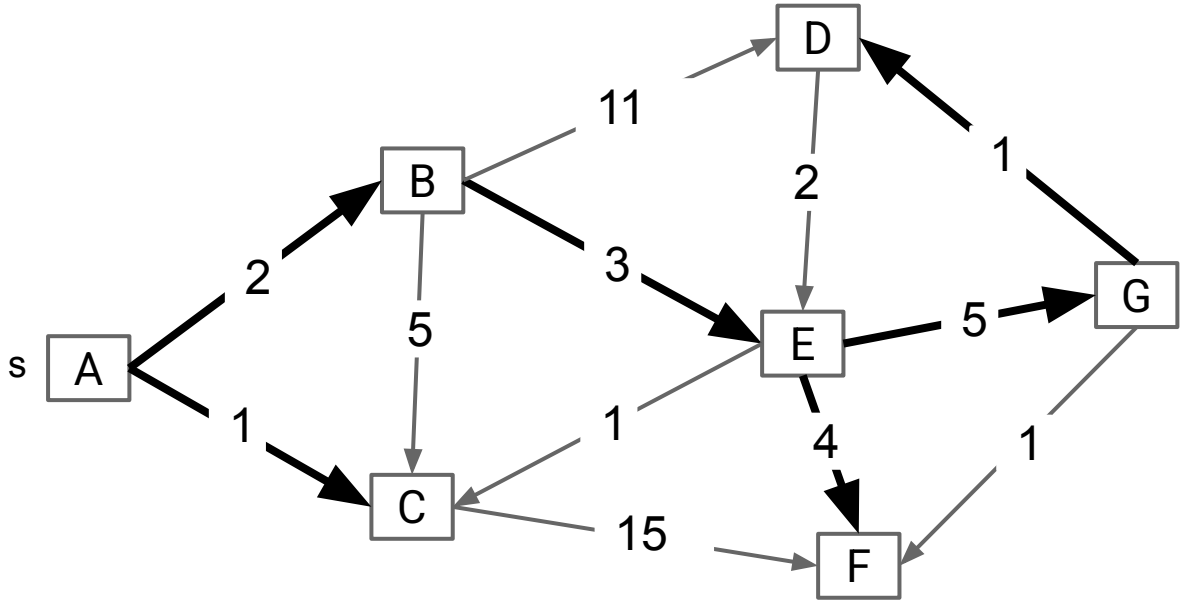
Note: If non-negative weights, **impossible for any inactive vertex** (white, not on fringe) **to be improved**!

Dijkstra's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from source.

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E



Fringe: []

Runtime Analysis

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- Why BFS Doesn't Work
- Goal: The Shortest Paths Tree

Dijkstra's Algorithm

- Dijkstra's Algorithm
- **Runtime Analysis**

A* (Extra, not in scope)

- A* Idea and Demo
- A* Heuristics (CS188 Preview)

Dijkstra's Algorithm Runtime

Priority Queue operation count, assuming binary heap based PQ:

- add: V, each costing O(log V) time.
- removeSmallest: V, each costing O(log V) time.
- changePriority: E, each costing O(log V) time.

Overall runtime: O(V*log(V) + V*log(V) + E*logV).

- Assuming E > V, this is just O(E log V) for a connected graph.

	# Operations	Cost per operation	Total cost
PQ add	V	O(log V)	O(V log V)
PQ removeSmallest	V	O(log V)	O(V log V)
PQ changePriority	E	O(log V)	O(E log V)

A* Idea and Demo

Lecture 23, CS61B, Spring 2026

Shortest Paths:

- Why BFS Doesn't Work
- Goal: The Shortest Paths Tree

Dijkstra's Algorithm

- Dijkstra's Algorithm
- Runtime Analysis

A* (Extra, not in scope)

- **A* Idea and Demo**
- A* Heuristics (CS188 Preview)

Single Target Dijkstra's

Is this a good algorithm for a navigation application?

- Will it find the shortest path?
- Will it be efficient?

The Problem with Dijkstra's

Dijkstra's will explore every place within nearly two thousand miles of Denver before it locates NYC.

The Problem with Dijkstra's

We have only a ***single target*** in mind, so we need a different algorithm. How can we do better?

How can we do Better?

Explore eastwards first?

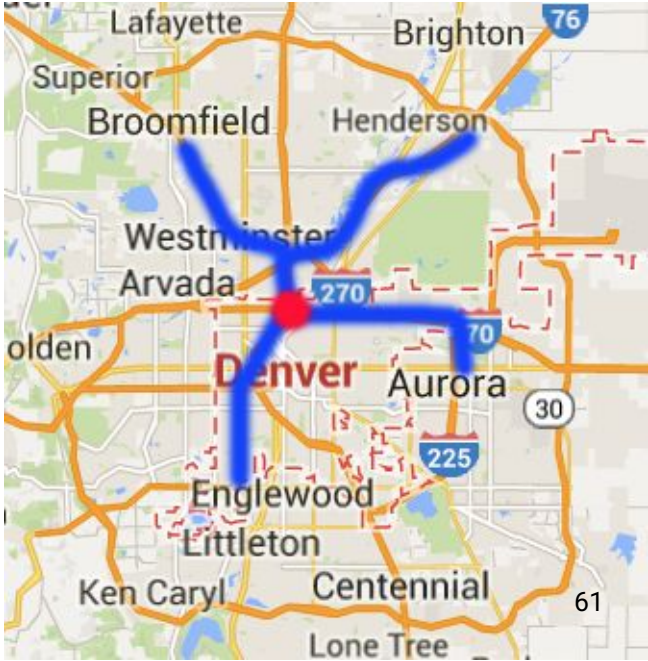
Introducing A*

Compared to Dijkstra's which only considers d(source, v).

Simple idea:

- Visit vertices in order of d(Denver, v) + h(v, goal), where h(v, goal) is an estimate of the distance from v to our goal NYC.
- In other words, look at some location v if:
    - We know already know the fastest way to reach v.
    - AND we suspect that v is also the fastest way to NYC taking into account the time to get to v.

Example: Henderson is farther than Englewood, but probably overall better for getting to NYC.

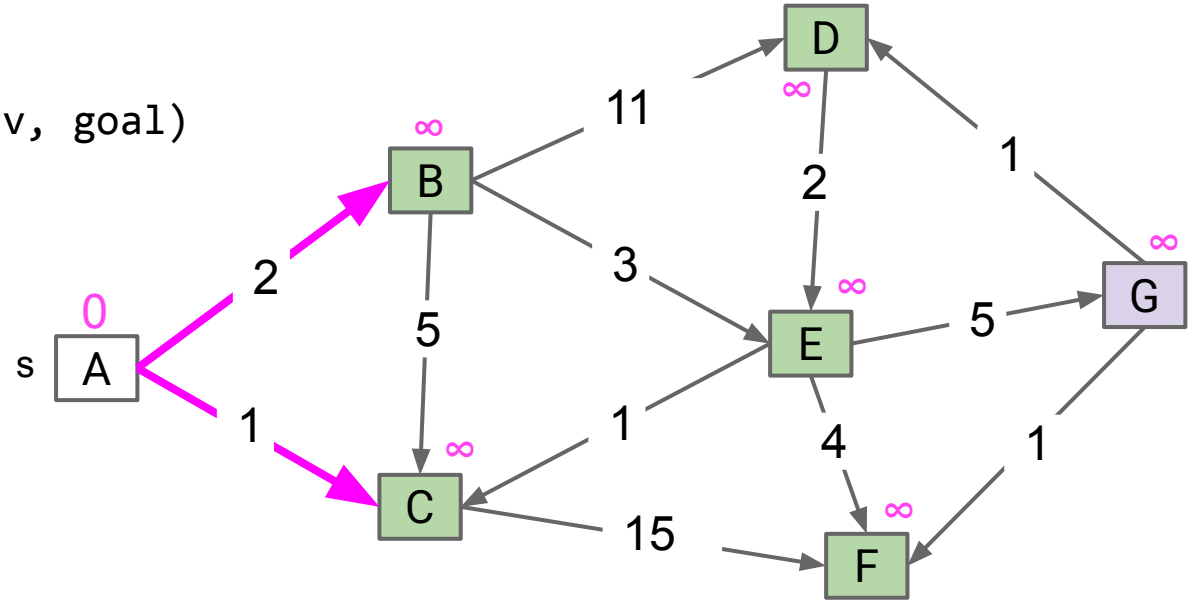


A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	∞	-	3
C	∞	-	15
D	∞	-	1
E	∞	-	1
F	∞	-	∞
G	∞	-	0



h(v, goal) is arbitrary. In this example, it's the min weight edge out of each vertex.

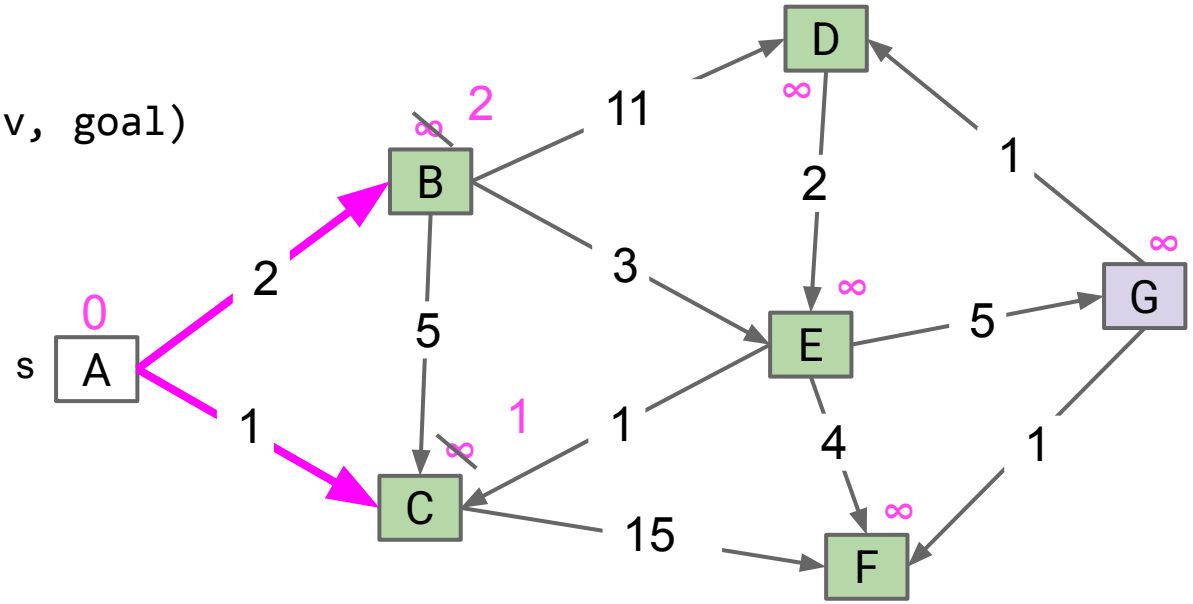
Fringe: [(B: ∞), (C: ∞), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	∞	-	1
E	∞	-	1
F	∞	-	∞
G	∞	-	0



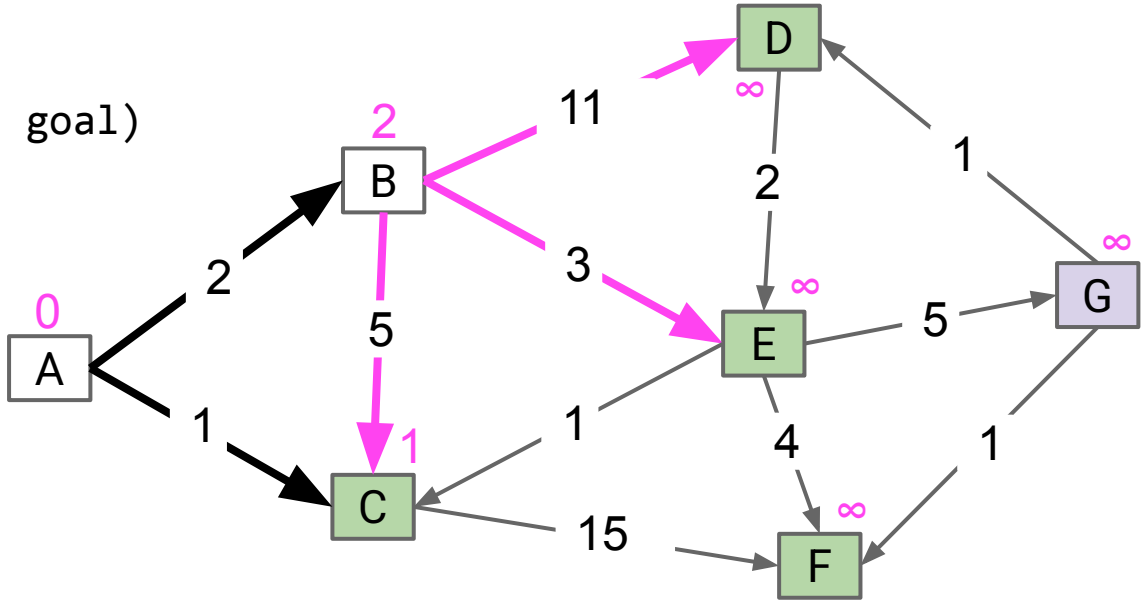
Fringe: [(B: 5), (C: 16), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	∞	-	2
E	∞	-	1
F	∞	-	∞
G	∞	-	0



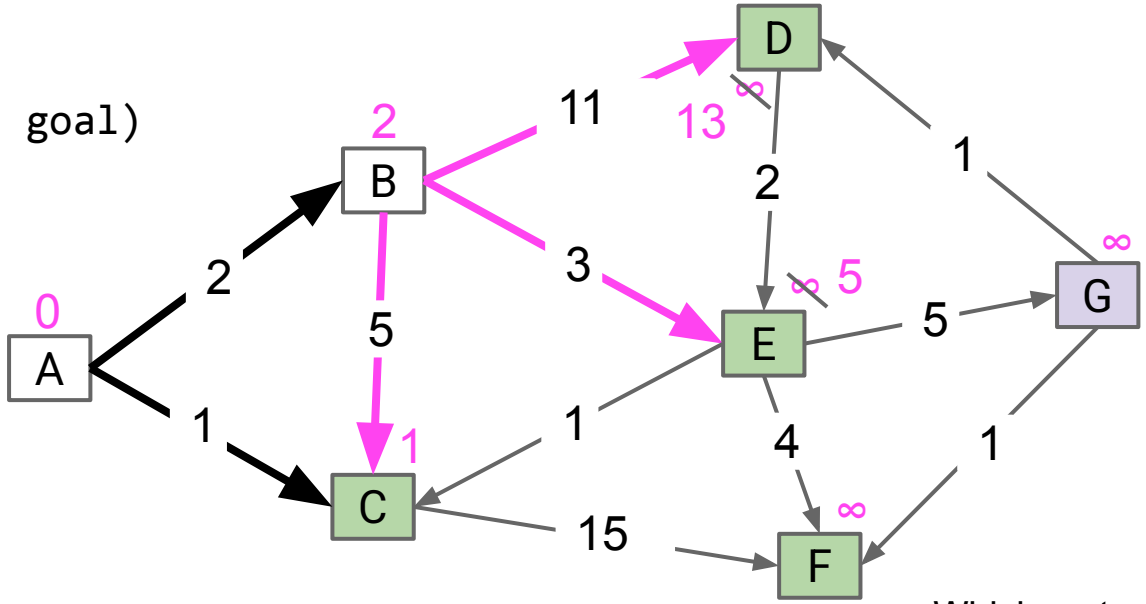
Fringe: [(C: 16), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	13	B	2
E	5	B	1
F	∞	-	∞
G	∞	-	0



Fringe: [(E: 6), (D: 15), (C: 16), (F: ∞), (G: ∞)]

Which vertex is removed next?

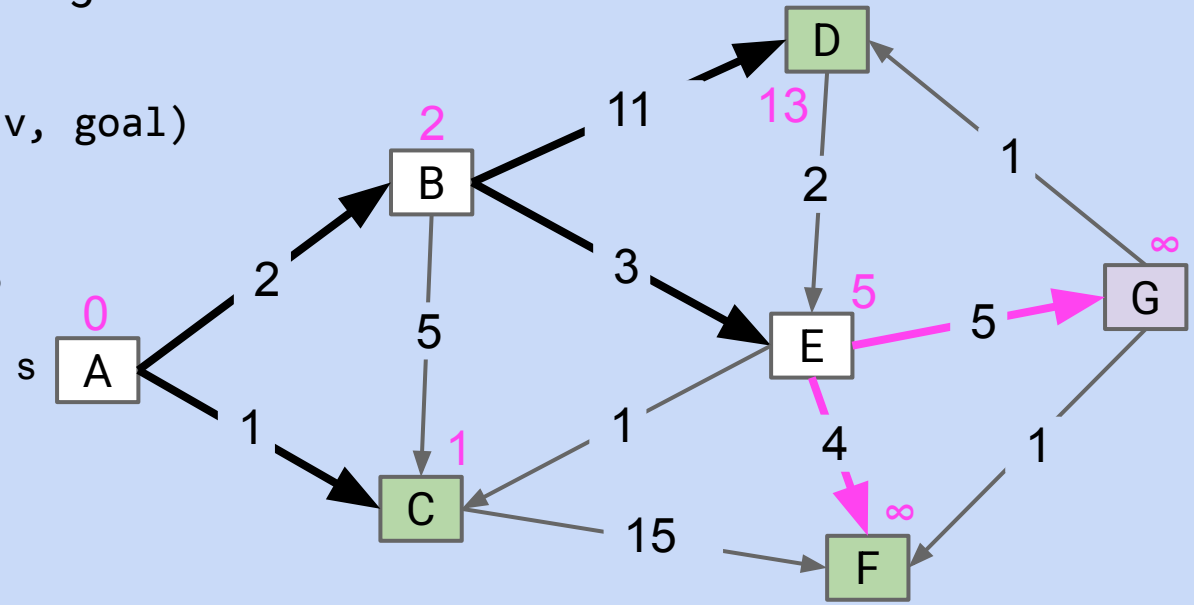
A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

- Give distTo, edgeTo, and fringe after relaxation

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	13	B	2
E	5	B	1
F	∞	-	∞
G	∞	-	0



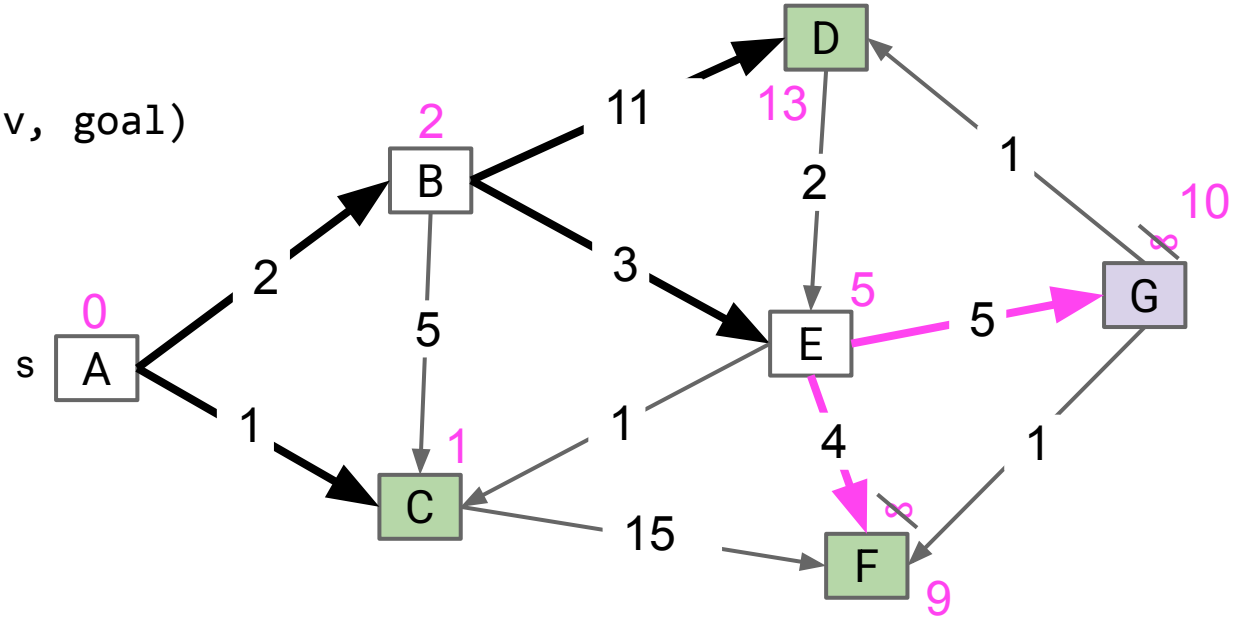
Fringe: [(D: 15), (C: 16), (F: ∞), (G: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	13	B	2
E	5	B	1
F	9	E	∞
G	10	E	0



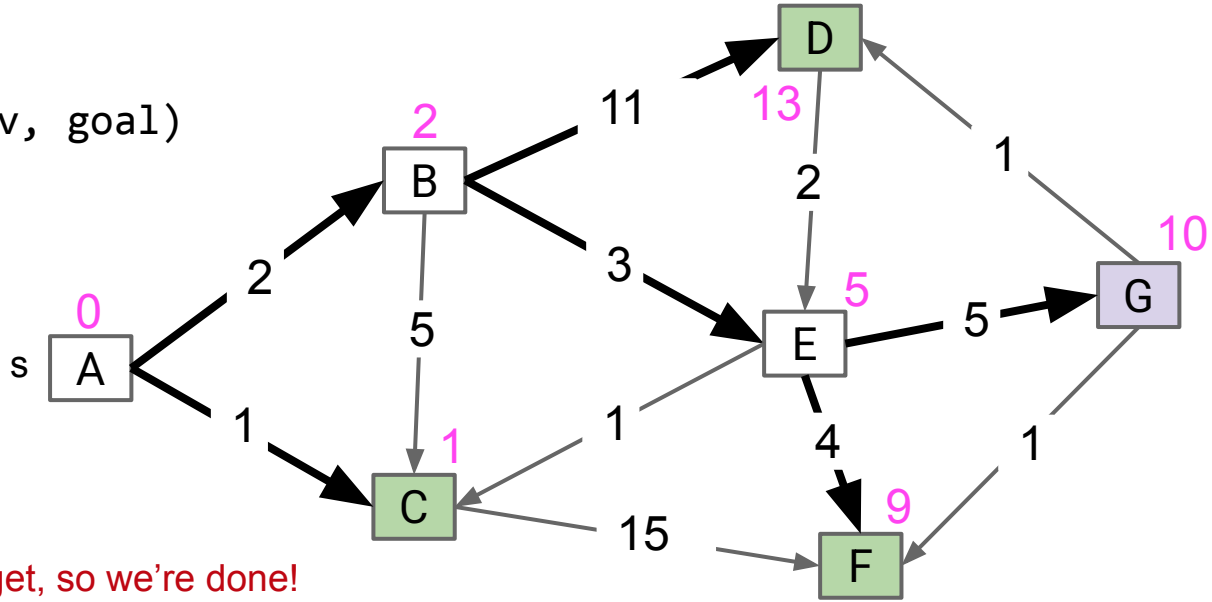
Fringe: [(G: 10), (D: 15), (C: 16), (F: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	13	B	2
E	5	B	1
F	9	E	∞
G	10	E	0



Next vertex to be dequeued is our target, so we're done!

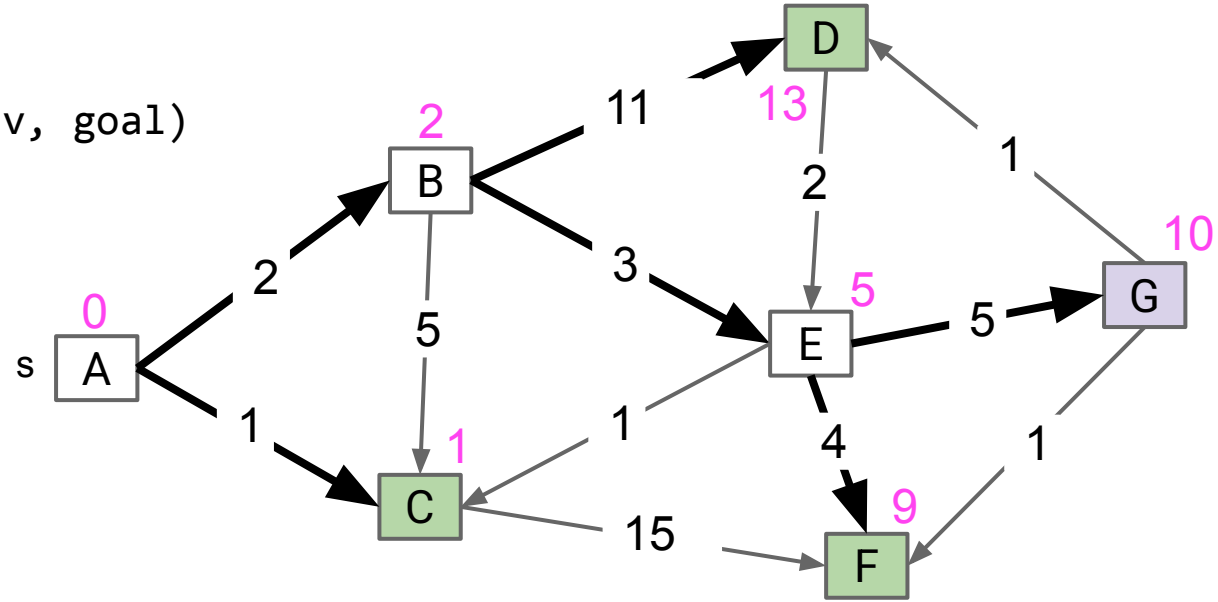
Fringe: [(G: 10), (D: 15), (C: 16), (F: ∞)]

A* Demo, with s = 0, goal = 6.

Insert all vertices into fringe PQ, storing vertices in order of d(source, v) + h(v, goal).

Repeat: Remove best vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo	h(v, goal)
A	0	-	1
B	2	A	3
C	1	A	15
D	13	B	2
E	5	B	1
F	9	E	∞
G	10	E	0



Observations:

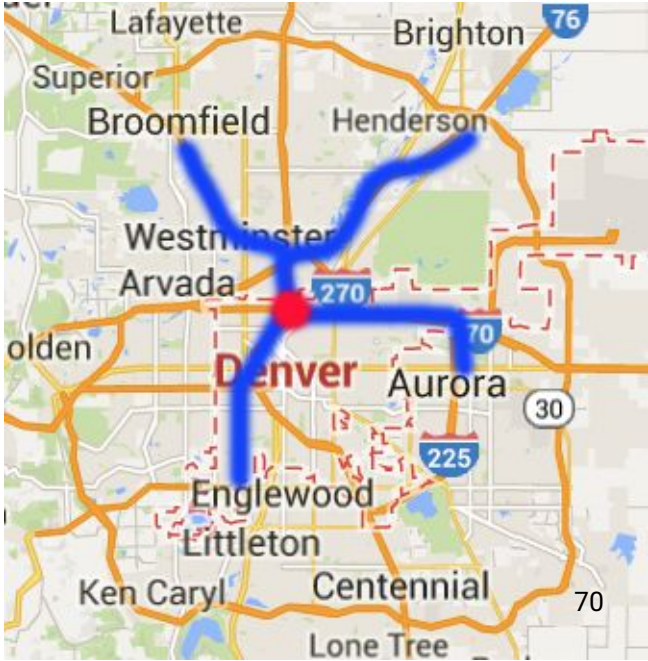
- Not every vertex got visited.
- Result is not a shortest paths tree for vertex A (path to D is suboptimal!), but that's OK because we only care about path to G.

A* Heuristic Example

How do we get our estimate?

- Estimate is an arbitrary ***heuristic*** h(v, goal).
- heuristic: “using experience to learn and improve”
- Doesn’t have to be perfect!

For the map to the right, what could we use?



A* Heuristic Example

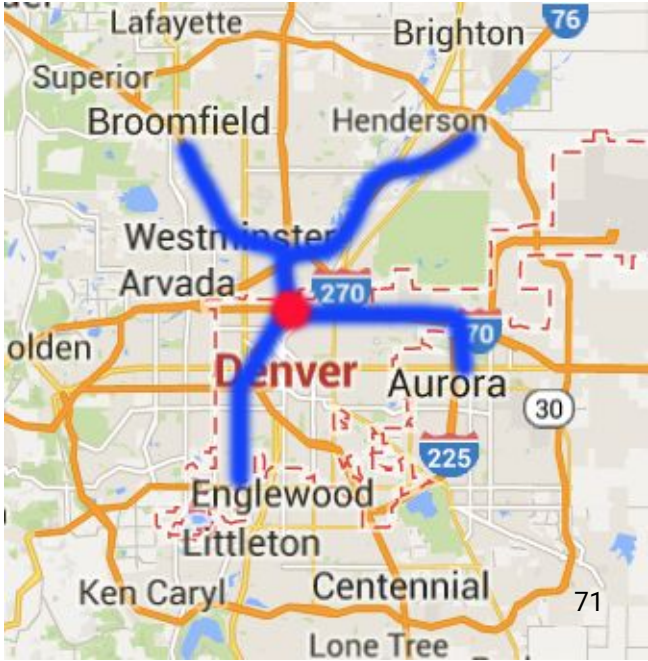
How do we get our estimate?

- Estimate is an arbitrary ***heuristic*** h(v, goal).
- heuristic: “using experience to learn and improve”
- Doesn’t have to be perfect!

For the map to the right, what could we use?

- As-the-crow-flies distance to NYC.

```
/** h(v, goal) DOES NOT CHANGE as algorithm runs. */
public method h(v, goal) {
   return computeLineDistance(v.latLong, goal.latLong);
}
```



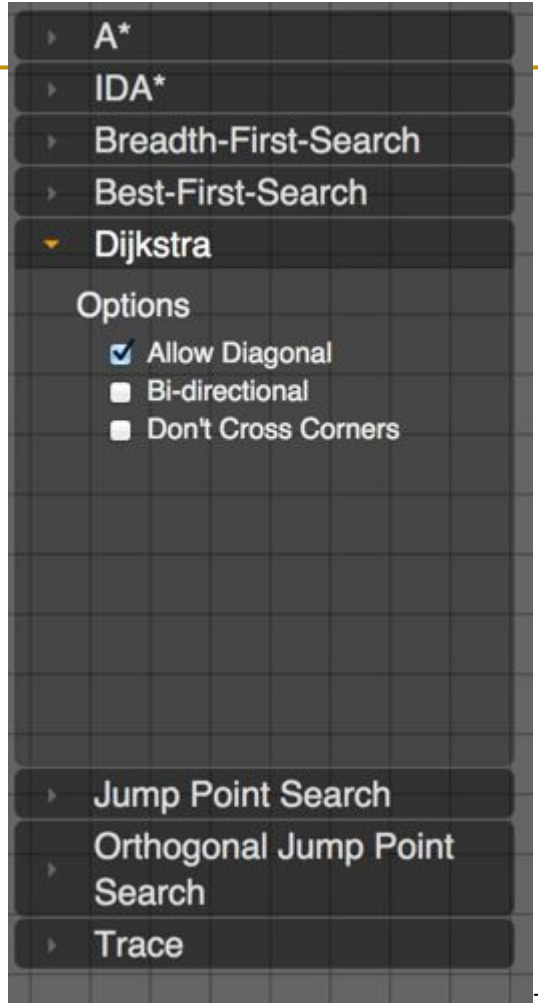
A* vs. Dijkstra's Algorithm

http://qiao.github.io/PathFinding.js/visual/

Note, if edge weights are all equal (as here), Dijkstra's algorithm is just breadth first search.

This is a good tool for understanding distinction between order in which nodes are visited by the algorithm vs. the order in which they appear on the shortest path.

- Unless you're really lucky, vastly more nodes are visited than exist on the shortest path.



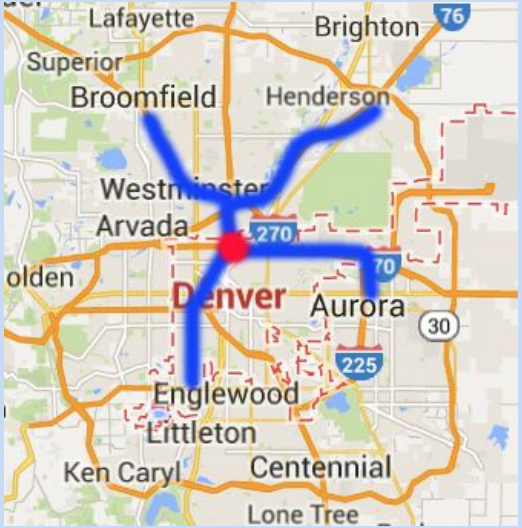
A* Heuristics (CS188 Preview)

Lecture 23, CS61B, Spring 2026

Impact of Heuristic Quality

Suppose we throw up our hands and say we don't know anything, and just set h(v, goal) = 0 miles. What happens?

What if we just set h(v, goal) = 10000 miles?



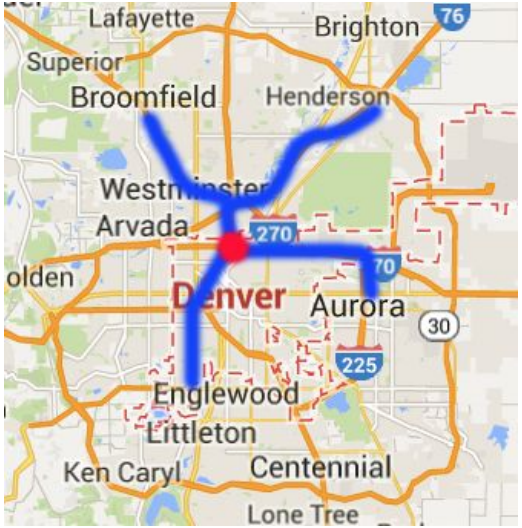
A* Algorithm:

Visit vertices in order of d(Denver, v) + h(v, goal), where h(v, goal) is an estimate of the distance from v to NYC.

Impact of Heuristic Quality

Suppose we throw up our hands and say we don't know anything, and just set h(v, goal) = 0 miles. What happens?

- We just end up with Dijkstra's algorithm.



What if we just set h(v, goal) = 10000 miles?

- We just end up with Dijkstra's algorithm.

A* Algorithm:

Visit vertices in order of d(Denver, v) + h(v, goal), where h(v, goal) is an estimate of the distance from v to NYC.

Impact of Heuristic Quality

Suppose you use your impressive geography knowledge and decide that the midwestern states of Illinois and Indiana are in the middle of nowhere: h(Indianapolis, goal)=h(Chicago, goal)=...=100000.

- Is our algorithm still correct or does it just run slower?

Impact of Heuristic Quality

Suppose you use your impressive geography knowledge and decide that the midwestern states of Illinois and Indiana are in the middle of nowhere: h(Indianapolis, goal)=h(Chicago, goal)=...=100000.

- Is our algorithm still correct or does it just run slower?
  - It is incorrect. It will fail to find the shortest path by dodging Illinois.

Heuristics and Correctness (EXTRA: Beyond Course Scope)

For our version of A* to give the correct answer, our A* heuristic must be:

- **Admissible**: h(v, NYC) ≤ true distance from v to NYC.
- **Consistent**: For each neighbor of w:
  - h(v, NYC) ≤ dist(v, w) + h(w, NYC).
  - Where dist(v, w) is the weight of the edge from v to w.

Our heuristic was inadmissible and inconsistent.

This is an artificial intelligence topic, and is beyond the scope of our course.

- We will not discuss these properties beyond their definitions. See CS188 which will cover this topic in considerably more depth.
- You should simply know that the **choice of heuristic matters**, and that if you make a **bad choice**, **A* can give the wrong answer.**
- You will not be expected to tell us whether a given heuristic is admissible or consistent unless we define these terms on an exam.

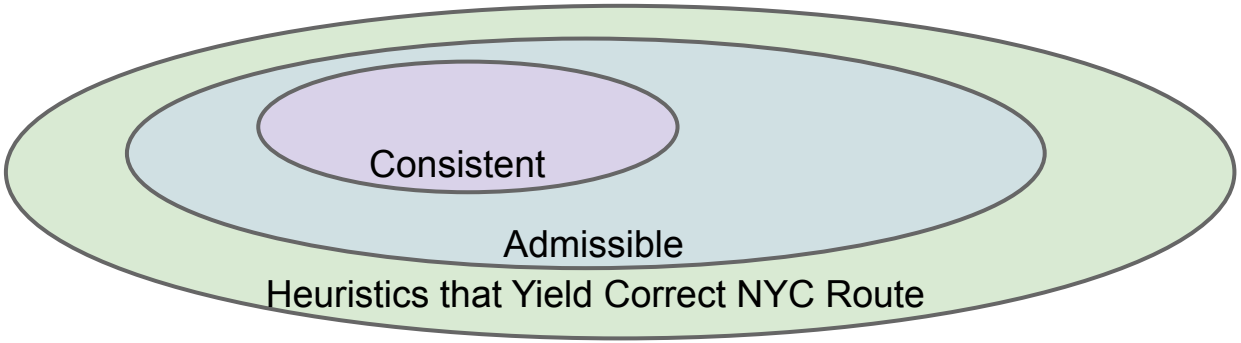
Consistency and Admissibility (EXTRA: Beyond Course Scope)

All consistent heuristics are admissible.

- ‘Admissible’ means that the heuristic never overestimates.

Admissibility and consistency are sufficient conditions for certain variants of A*.

- If heuristic is admissible, A* tree search yields the shortest path.
- If heuristic is consistent, A* graph search yields the shortest path.
- These conditions are sufficient, but not necessary.



Our version of A* is called “A* graph search”. There’s another version called “A* tree search”. You’ll learn about it in 188.

Summary: Shortest Paths Problems

Single Source, Multiple Targets:

- Can represent shortest path from start to every vertex as a shortest paths tree with V-1 edges.
- Can find the SPT using Dijkstra's algorithm.

Single Source, Single Target:

- Dijkstra's is inefficient (searches useless parts of the graph).
- Can represent shortest path as path (with up to V-1 vertices, but probably far fewer).
- A* is potentially much faster than Dijkstra's.
  - Consistent heuristic guarantees correct solution.

Graph Problems

Problem	Problem Description	Solution	Efficiency
paths	Find a path from s to every reachable vertex.	DepthFirstPaths.java Demo	O(V+E) time Θ(V) space
shortest paths	Find the shortest path from s to every reachable vertex.	BreadthFirstPaths.java Demo	O(V+E) time Θ(V) space
shortest weighted paths	Find the shortest path, considering weights, from s to every reachable vertex.	DijkstrasSP.java Demo	O(E log V) time Θ(V) space
shortest weighted path (out of scope)	Find the shortest path, consider weights, from s to some target vertex (out of scope)	A*: Same as Dijkstra's but with h(v, goal) added to priority of each vertex. Demo (out of scope)	Time depends on heuristic. Θ(V) space (out of scope)